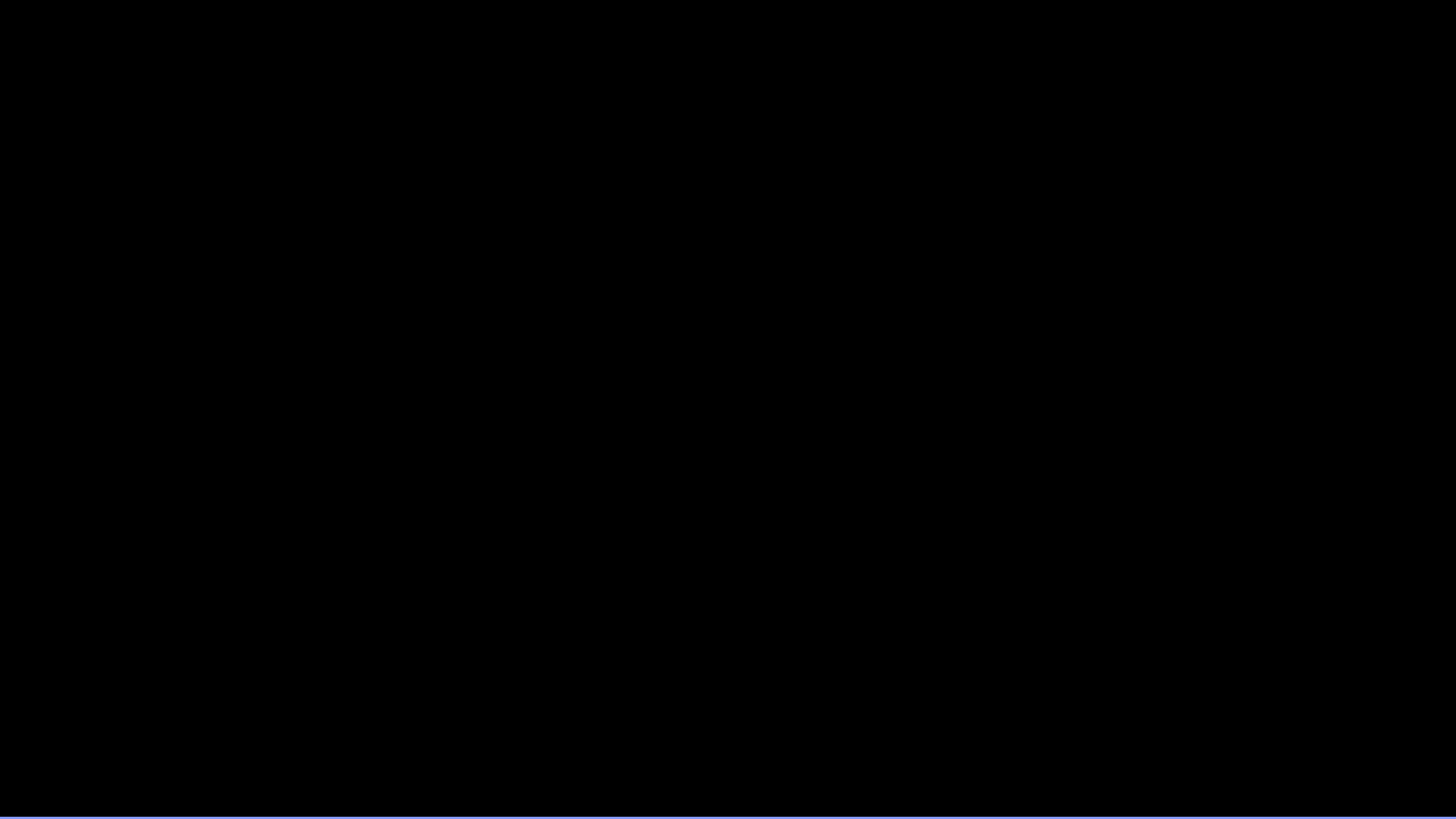






CNN (Convolutional Neural Network)

In mathematics, **convolution** is an integral operation that shows how another function modifies the shape of one function.

Convolution is a mathematical operation that simplifies complex math, physics or engineering operations.

CNN has been used in image and audio processing, NLP, and biomedical, but the best results are derived from **image processing**.



EXCLUSIVE



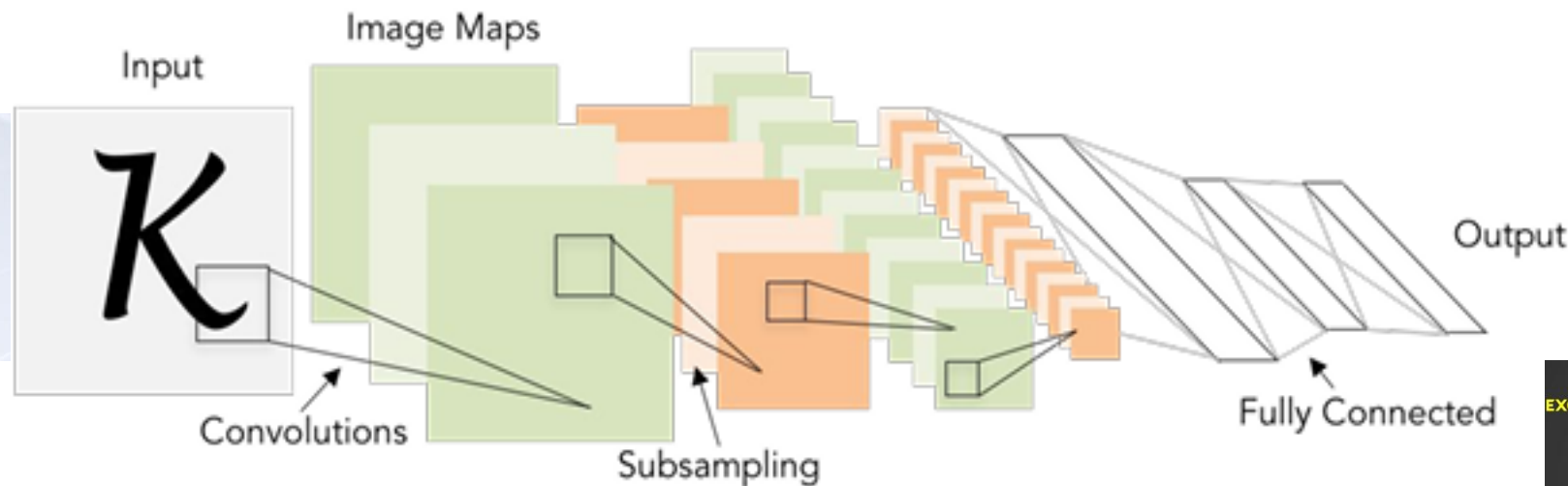
The GodFather of Deep Learning: **Yann LeCun**

- Yann LeCun is primarily known for his work on the **MNIST** dataset.
- MNIST is considered a **milestone** in image processing and deep learning.
- LeCun's work was an **essential step** in deep learning's exploration of the real world.

Convolutional Networks: LeCun et al, 1998



YANN LECUN
Facebook,
New York University

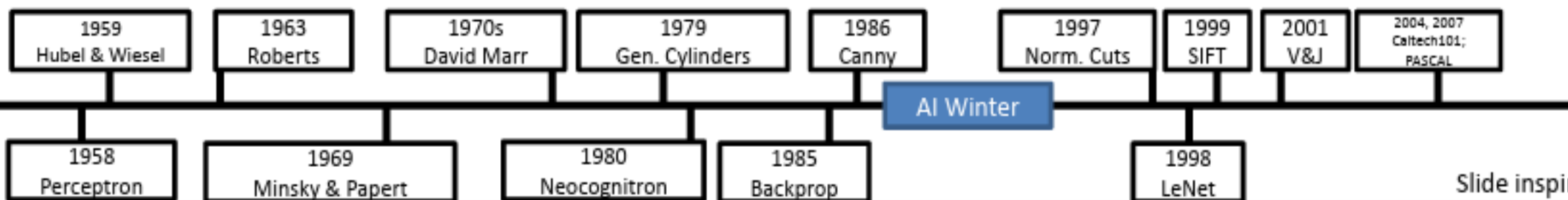


Applied backprop algorithm to a Neocognitron-like architecture

Learned to recognize handwritten digits

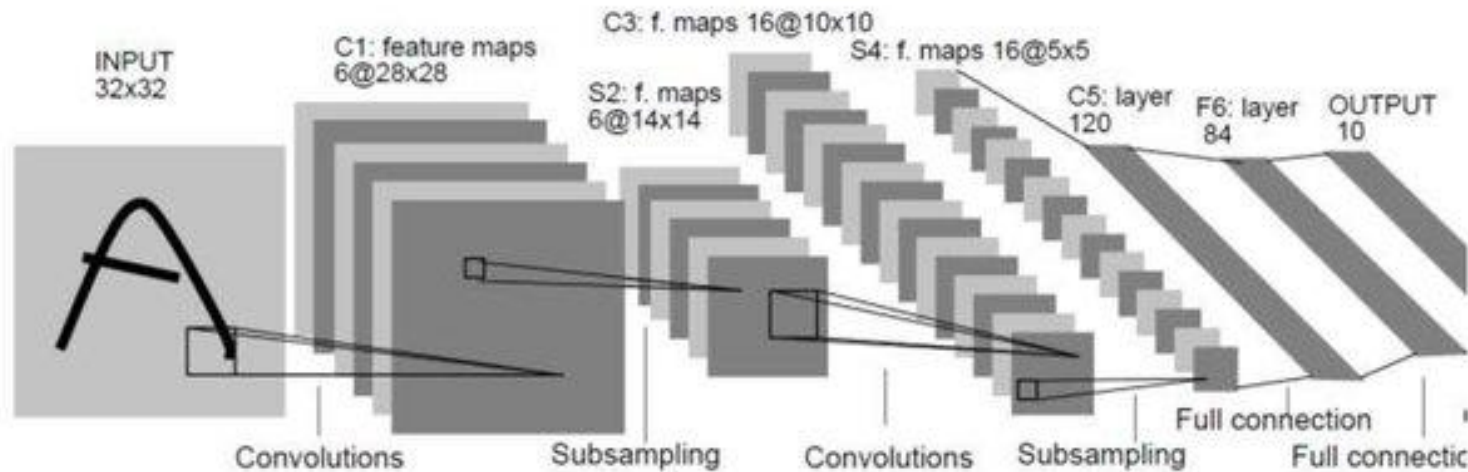
Was deployed in a commercial system by NEC, processed handwritten checks

Very similar to our modern convolutional networks!



Slide inspiration: Justin Johnson

Lenet-5 (1998)



MNIST: handwritten digits

- 70,000 28x28 pixel images
- Gray scale
- 10 classes



CIFAR-10: simple objects

- 60,000 32x32 pixel images
- RGB
- 10 classes

1989 (Lecun) A convnet is used for an image classification task (zip codes)

- First time backprop is used to automatically learn visual features
- Two convolutional layers, two fully connected layer (16x16 input, 12 FMs each layer, 5x5 filters)
- Stride=2 is used to reduce image dimensions
- Scaled Tanh activation function
- Uniform random weight initialization

1998 (Lecun) LeNet-5 convnet achieves state of the art result on MNIST

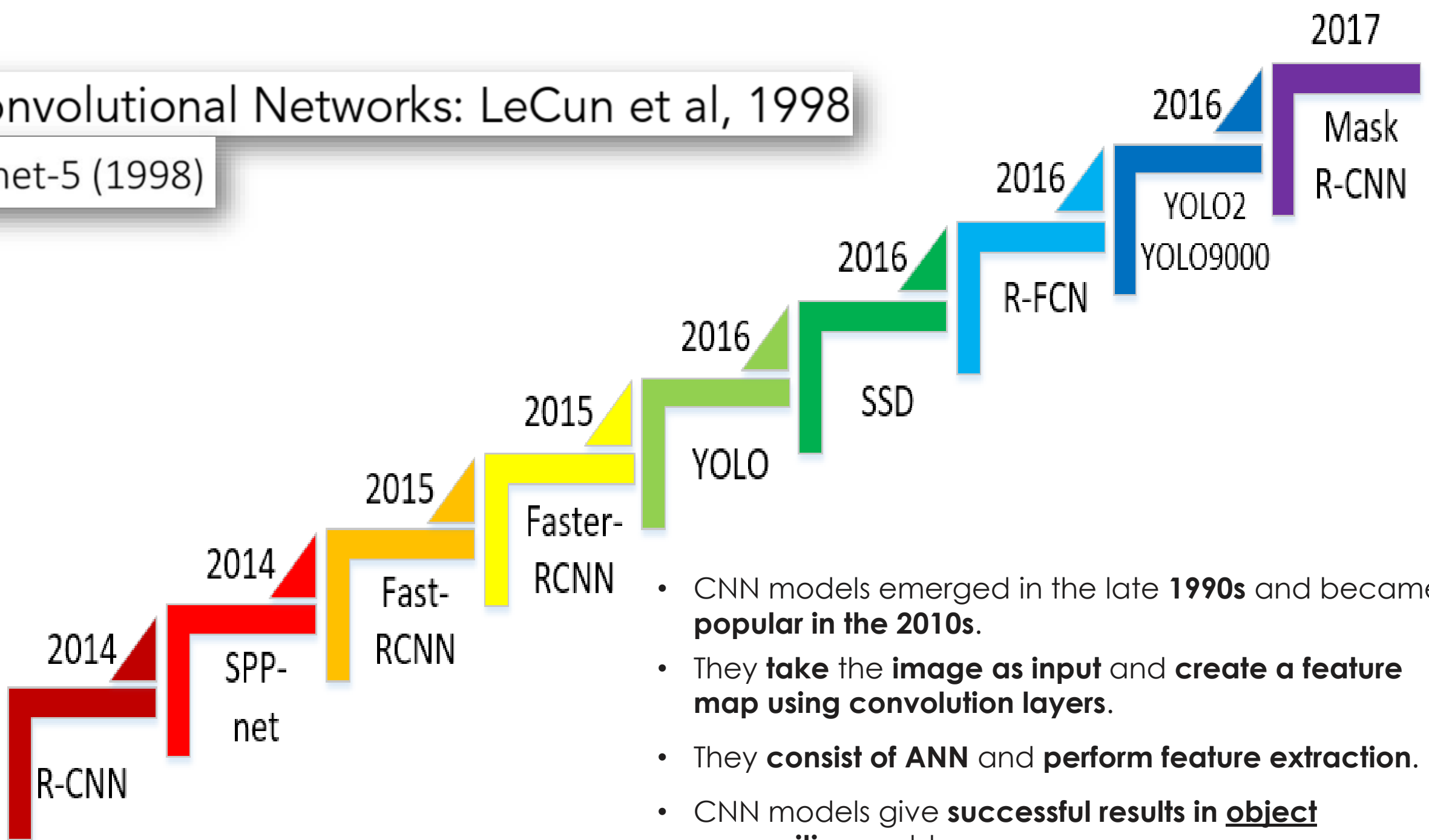
- Two convolutional layers, three fully connected layers (32x32 input, 6 and 12 FMs, 5x5 filters)
- Average pooling to reduce image dimensions
- Sparse connectivity between feature maps



How works LeNet?

Convolutional Networks: LeCun et al, 1998

Lenet-5 (1998)



- CNN models emerged in the late **1990s** and became **popular in the 2010s**.
- They **take the image as input** and **create a feature map using convolution layers**.
- They **consist of ANN** and **perform feature extraction**.
- CNN models give **successful results in object recognition** problems.

Model	Year	Developer(s)	Key Improvements
LeNet-5	1998	Yann LeCun	First modern CNN model, designed for character recognition . 5-layer architecture, used sigmoid and tanh activation functions .
AlexNet	2012	Alex Krizhevsky, Ilya Sutskever, Geoffrey Hinton	Larger and deeper architecture . Used ReLU activation for faster training. Showcased the potential of GPUs in deep learning.
ZFNet	2013	Matthew Zeiler, Rob Fergus	An extended and optimized version of AlexNet . Focused on visualizing filters and made changes in filter sizes and layers .
VGGNet	2014	Karen Simonyan, Andrew Zisserman	Built deep architectures (16 and 19 layers) with small (3x3) filters. Provided a uniform layer structure for deeper networks.
GoogLeNet (Inception)	2014	Christian Szegedy and others	Introduced the " Inception " module combining filters of different sizes. Achieved parameter efficiency and created deeper yet lighter models.
R-CNN (Region-based CNN)	2014	Ross Girshick and others	Proposed a region-based approach for object detection. Used CNN for feature extraction on each proposed region.
Fast R-CNN	2015	Ross Girshick	Improved R-CNN by using ROI pooling, increased speed, and accuracy. Applied CNN once for the whole image instead of region-wise.
Faster R-CNN	2015	Shaoqing Ren and others	Introduced Region Proposal Network (RPN), integrating the proposal stage into the CNN, significantly improving speed over previous R-CNN models.
ResNet	2015	Kaiming He and others	Introduced residual connections (skip connections) to enable training of much deeper networks (50, 101, 152 layers). Solved the vanishing gradient problem.
YOLO (You Only Look Once)	2016	Joseph Redmon and others	Processed the entire image in a single step, achieving very fast object detection. Optimized for real-time performance.
SSD (Single Shot Multibox Detector)	2016	Wei Liu and others	Similar to YOLO in terms of speed, but used feature maps at different scales, improving detection accuracy for objects of different sizes.
DenseNet	2016	Gao Huang and others	Connected each layer to all previous layers (dense connections), improving feature reuse and flow of information with fewer parameters.
Xception	2017	François Chollet	A deeper version of the "Inception" architecture, using depthwise separable convolutions for high performance with fewer parameters.
MobileNet	2017	Andrew G. Howard and others	Optimized for mobile devices. Reduced computational cost and parameters using depthwise separable convolutions.
NASNet	2018	Barret Zoph and others	Used automated machine learning (AutoML) to search for the best CNN architecture. Achieved performance for both small and large models.
EfficientNet	2019	Mingxing Tan, Quoc V. Le	Introduced a scaling strategy (depth, width, resolution) to provide a family of models that balance performance and efficiency.



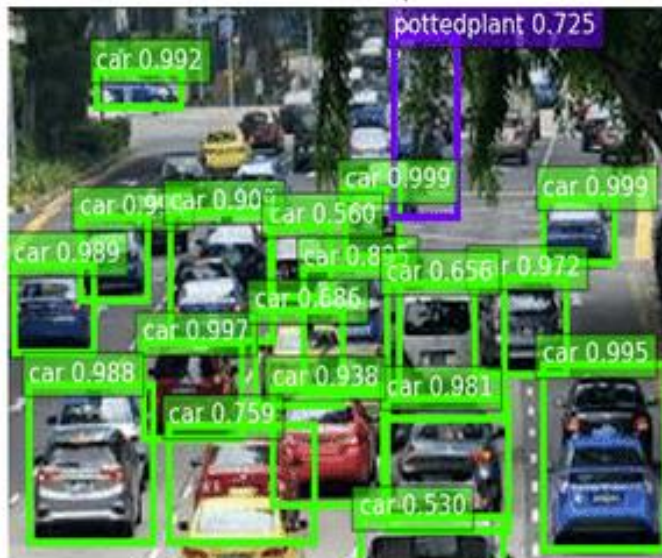
CNN

The input image

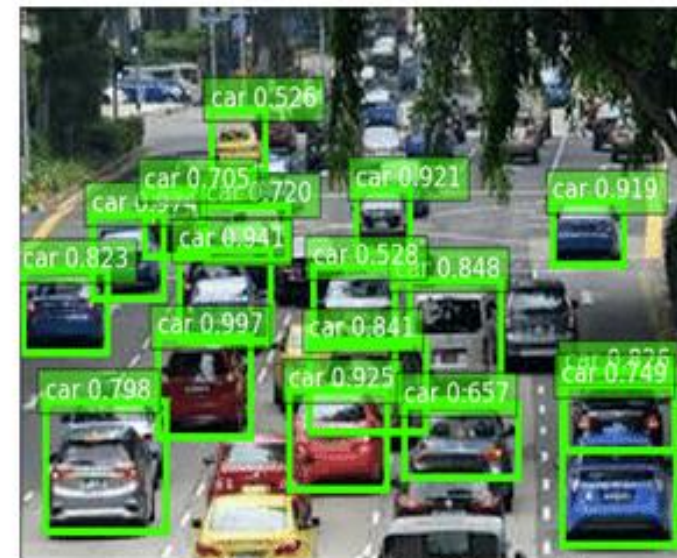


Faster
R-CNN

YOLO

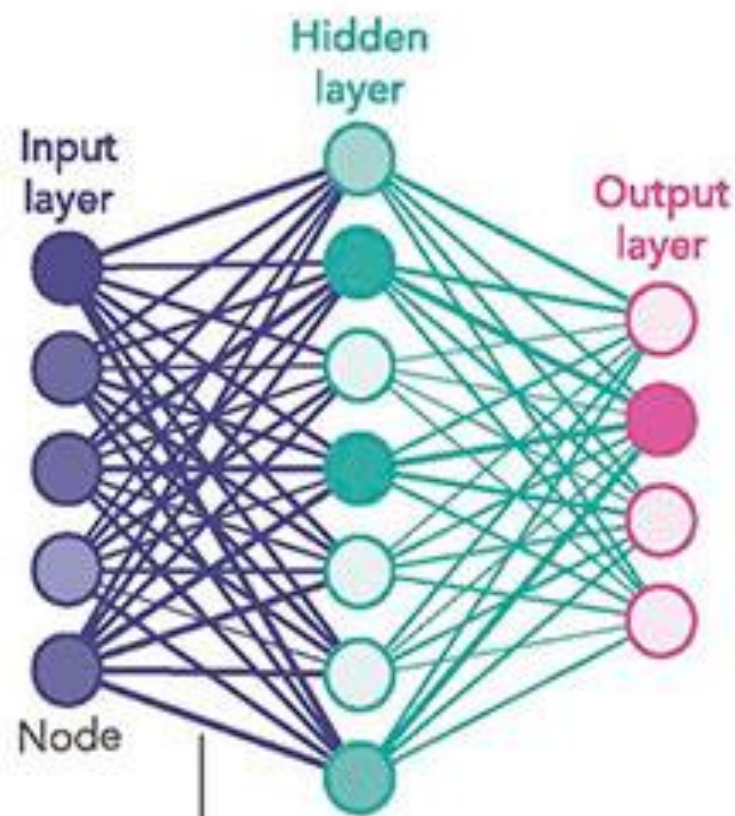


The predicted image by
Faster R-CNN



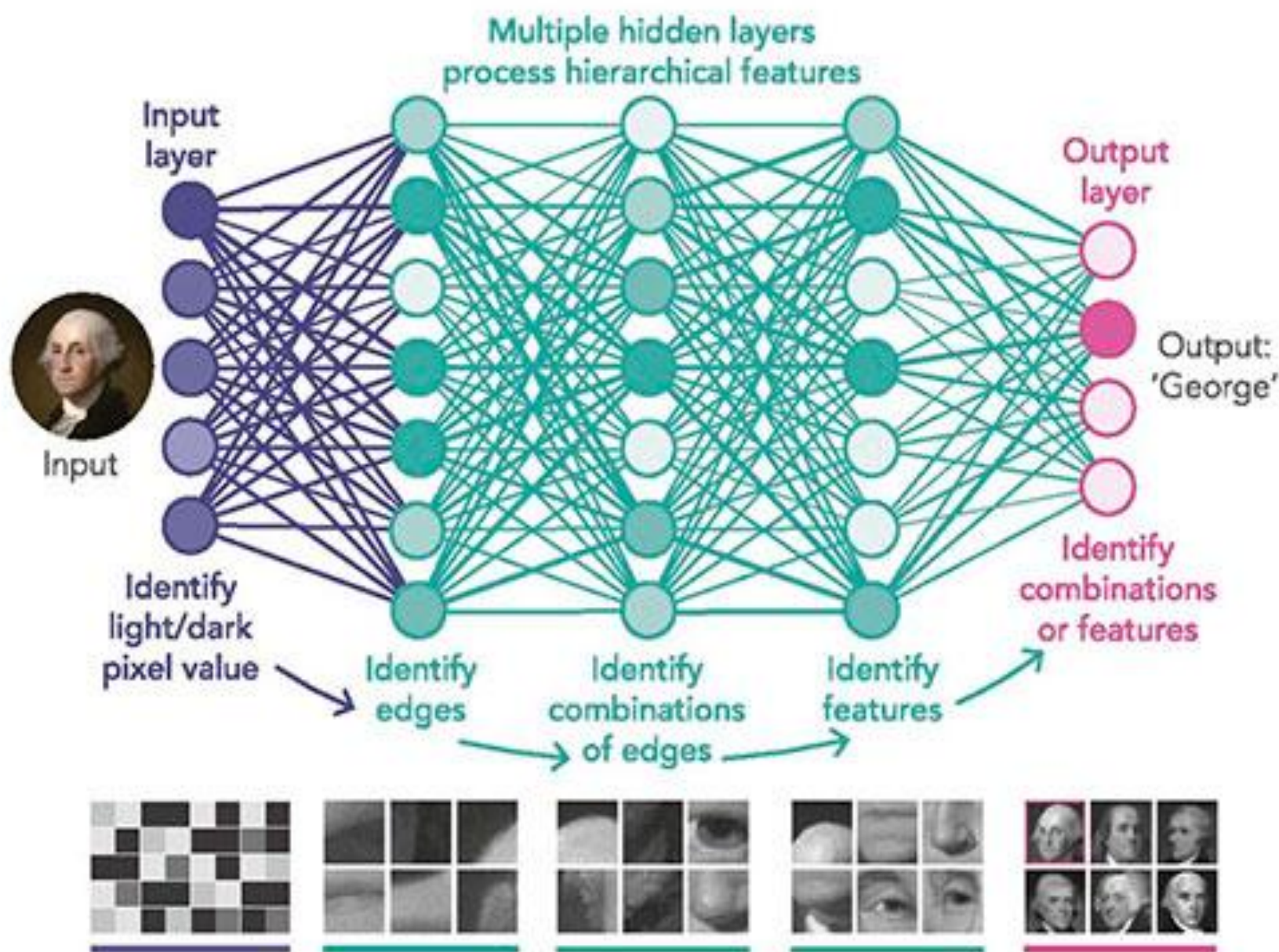
The predicted image by
YOLO

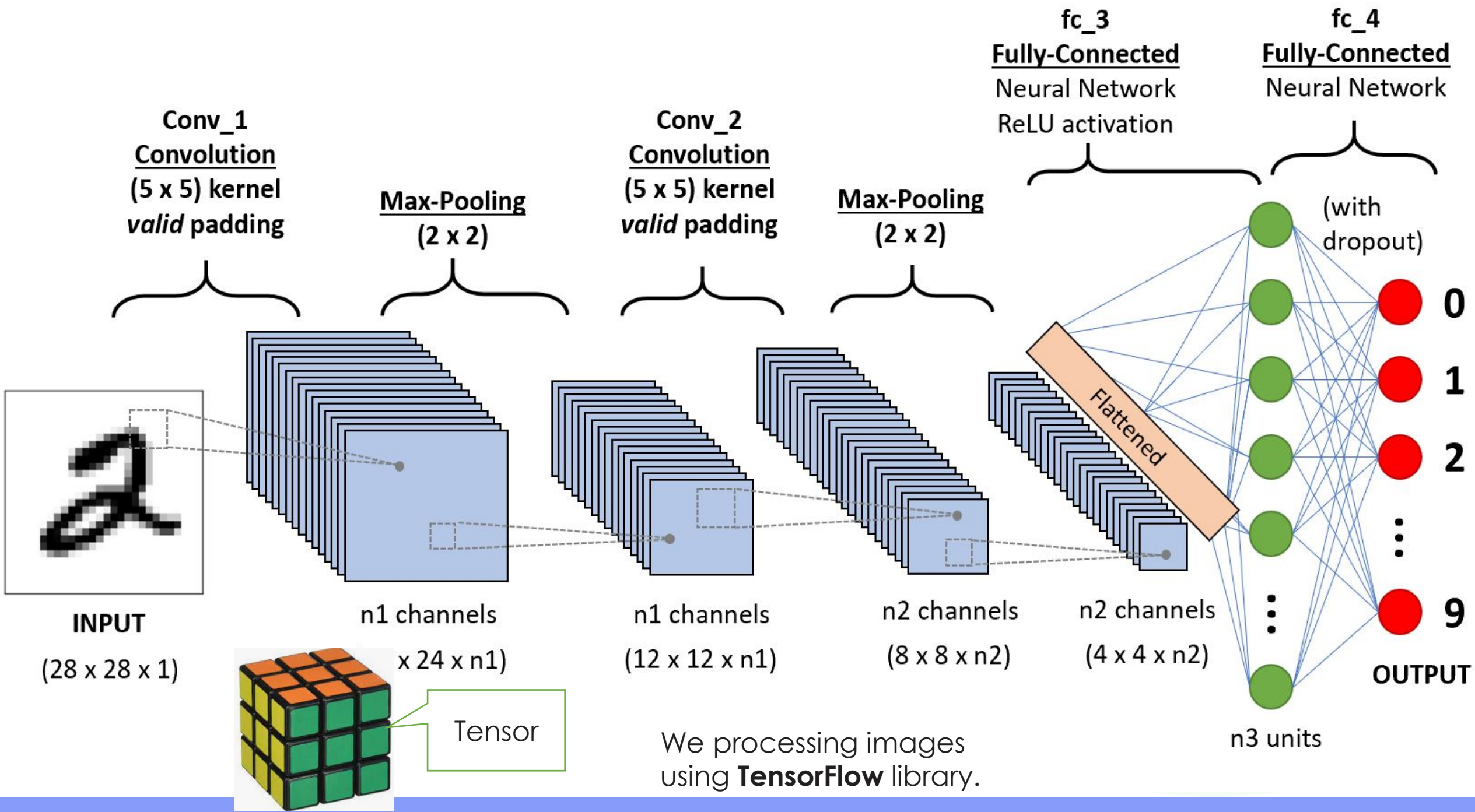
1980S-ERA NEURAL NETWORK



Links carry signals from one node to another, boosting or damping them according to each link's 'weight'.

DEEP LEARNING NEURAL NETWORK





Draw your number here

3



Downsampled drawing: 3

First guess: 3

Second guess: 1

Layer visibility

Input layer

Show

Convolution layer 1

Show

Downsampling layer 1

Show

Convolution layer 2

Show

0123456789

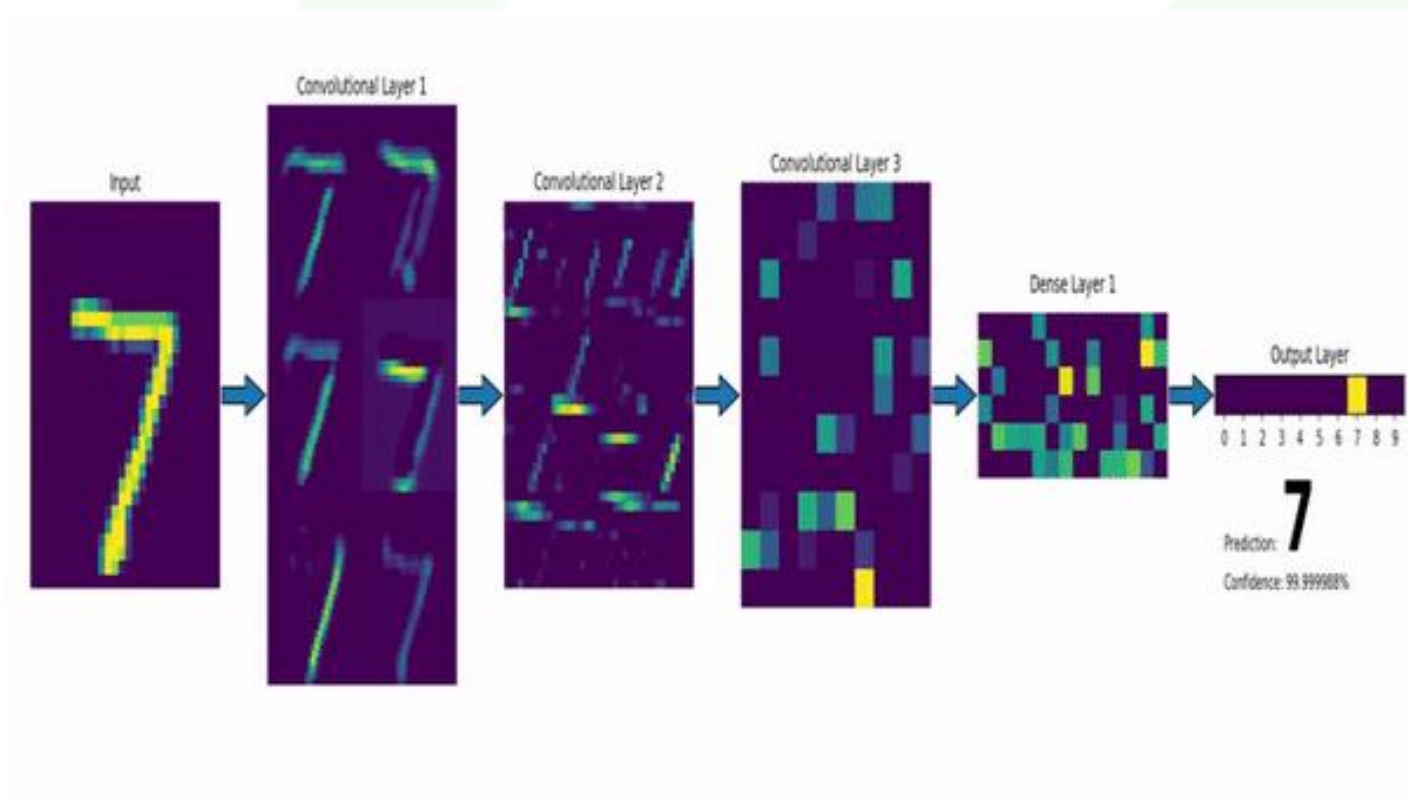
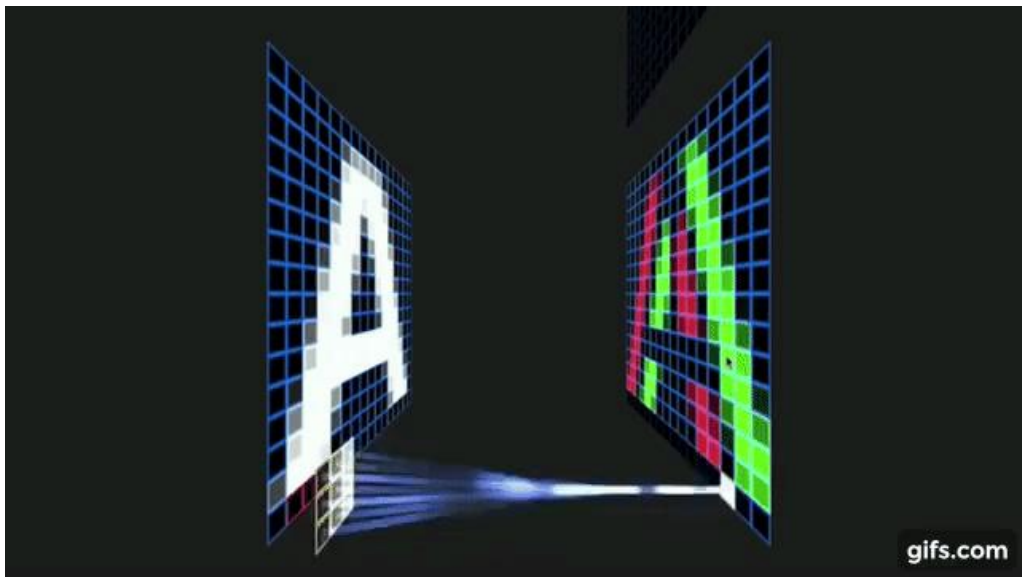


https://adamharley.com/nn_vis/cnn/2d.html

https://adamharley.com/nn_vis/cnn/3d.html

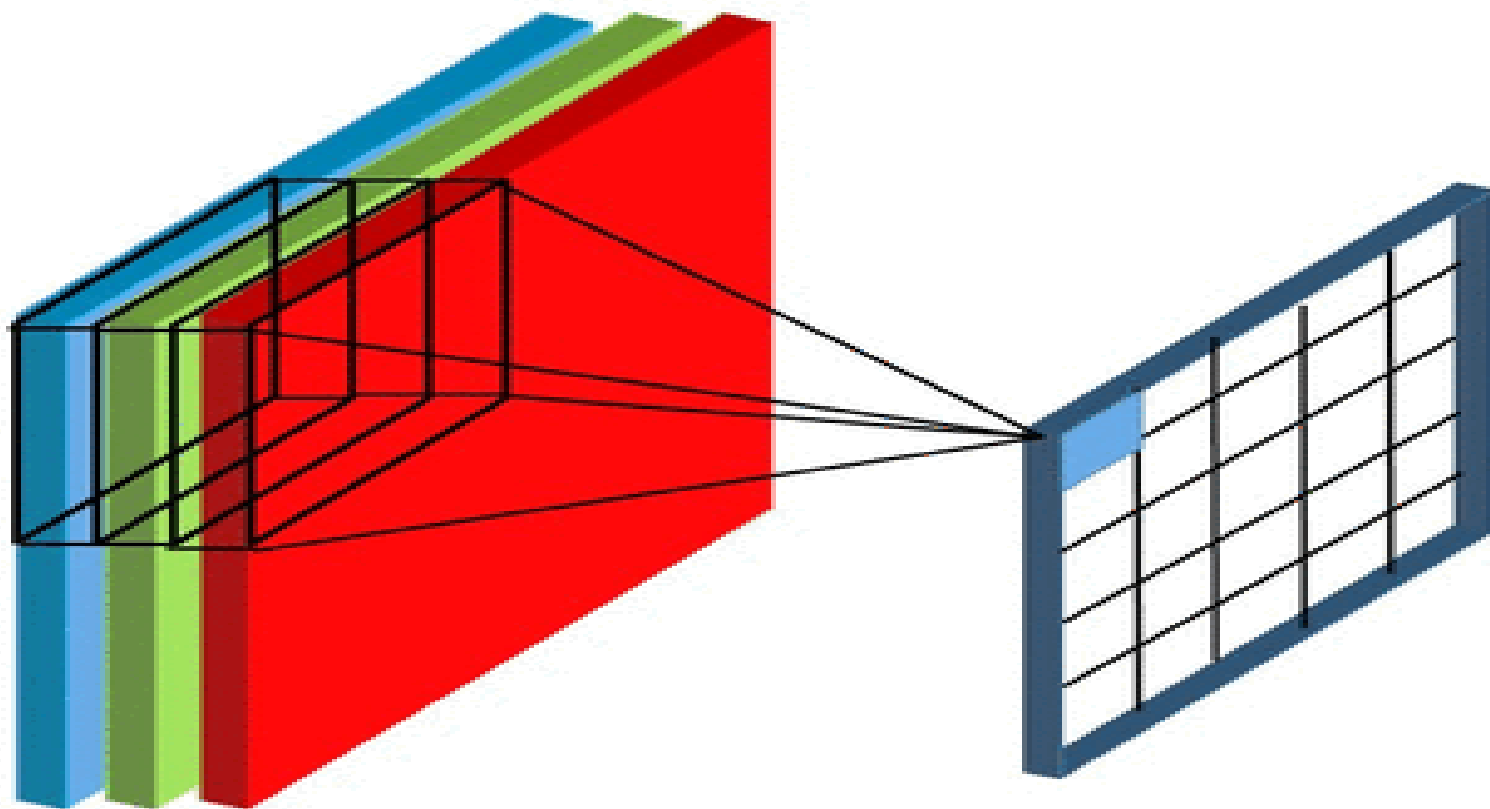


CNN



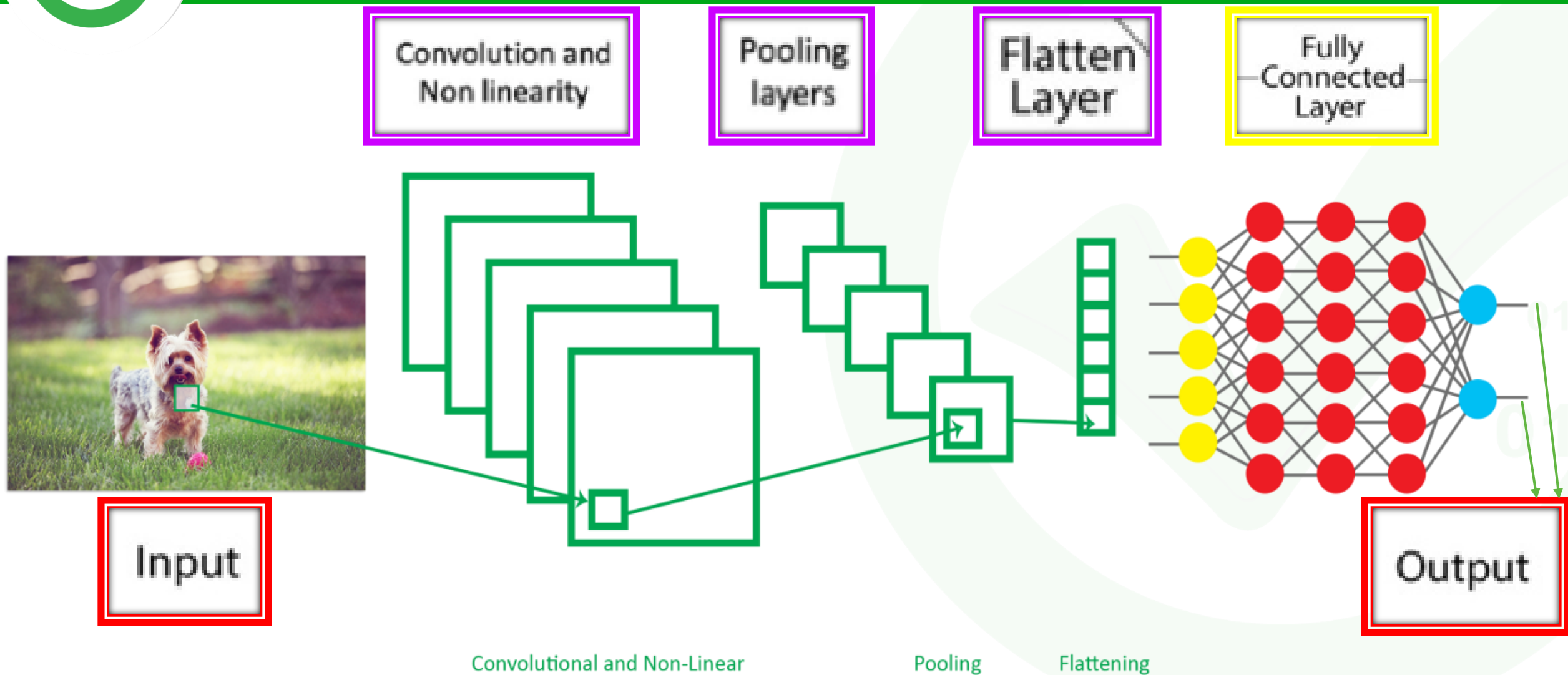


CNN





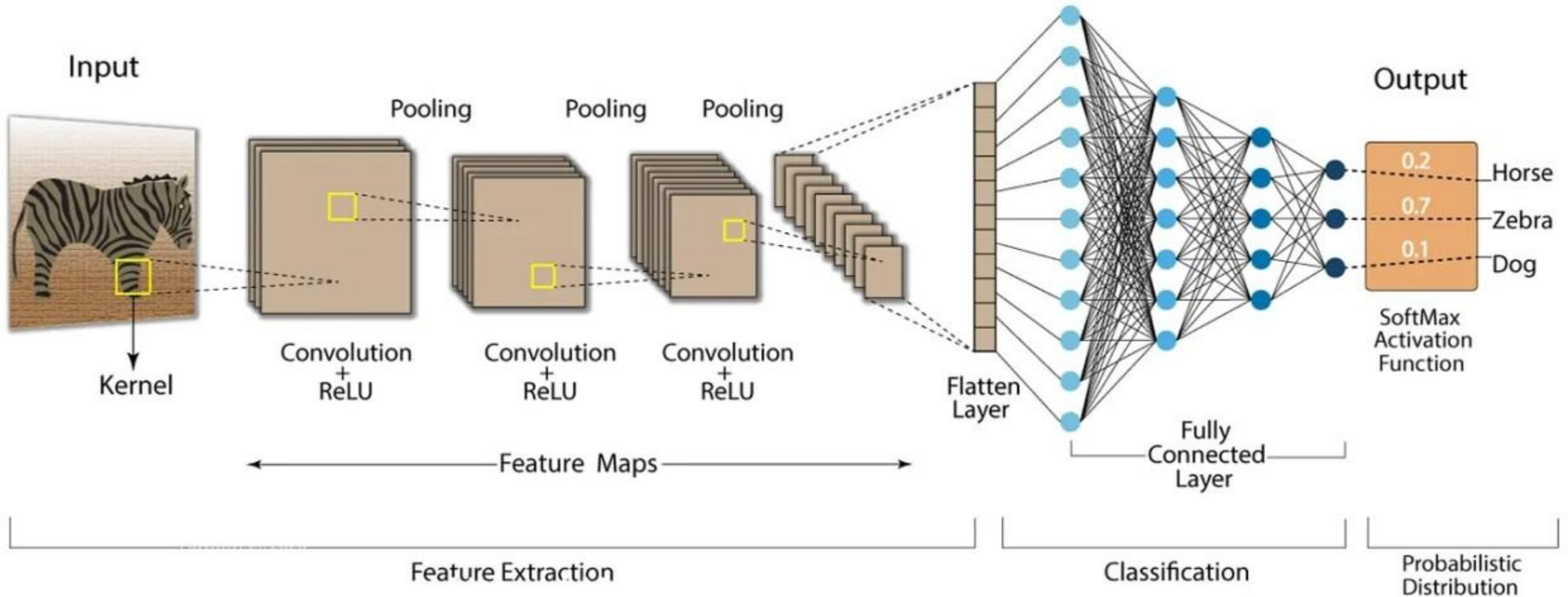
CNN





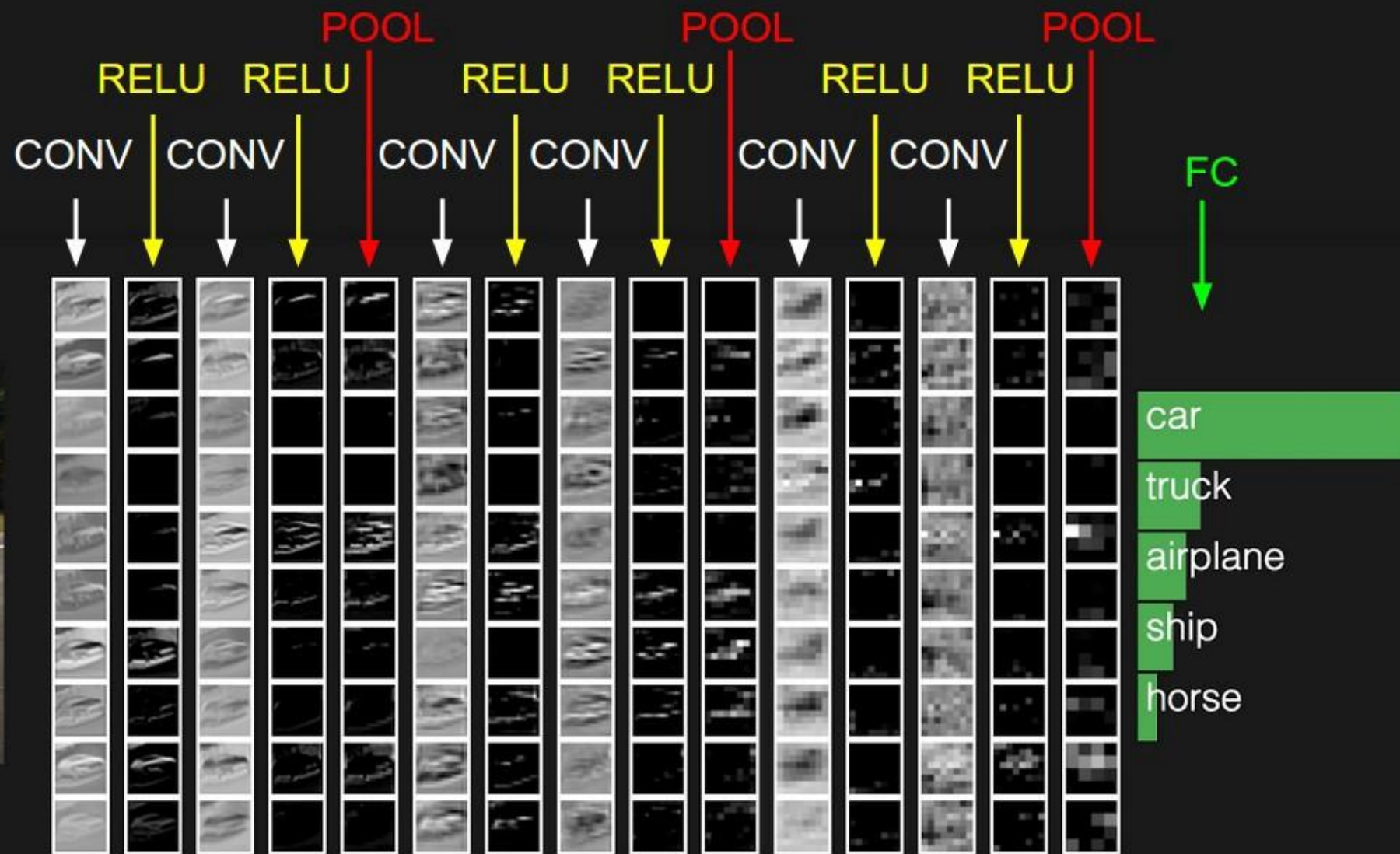
CNN

Convolution Neural Network (CNN)





CNN



DATE : 28.04.2026

DT/NT : DT

LESSON : DEEP LEARNING

**SUBJECT: CNN (CONVOLUTIONAL NEURAL
NETWORK) II**

BATCH : B322



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59





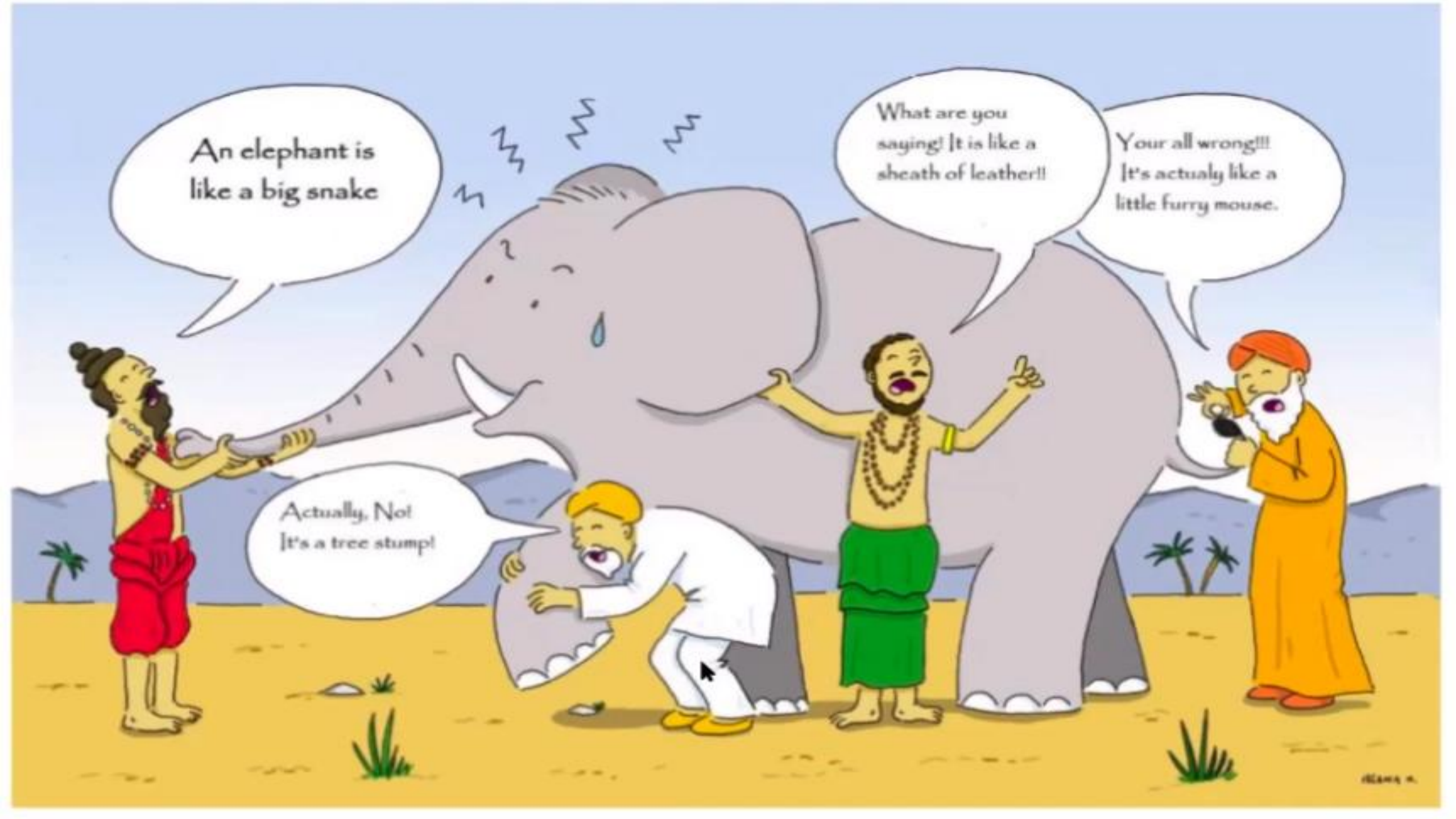
CONVOLUTIONAL LAYER



**AN EXCELLENT ILLUSTRATION OF
HOW CONVOLUTION NEURAL
NETWORKS (CNN) WORK!**



<https://www.youtube.com/shorts/N6NBT-n9mmo>

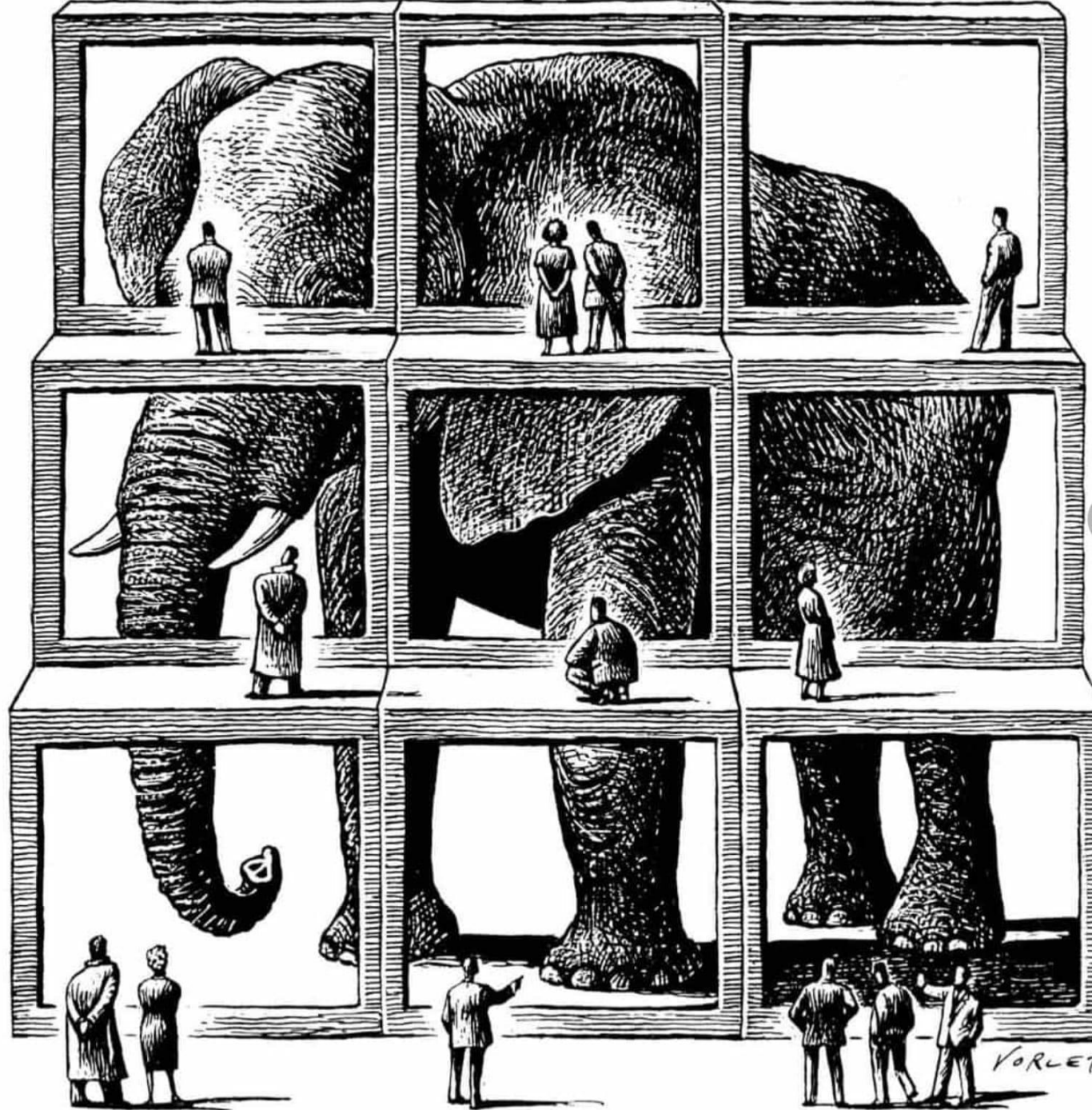
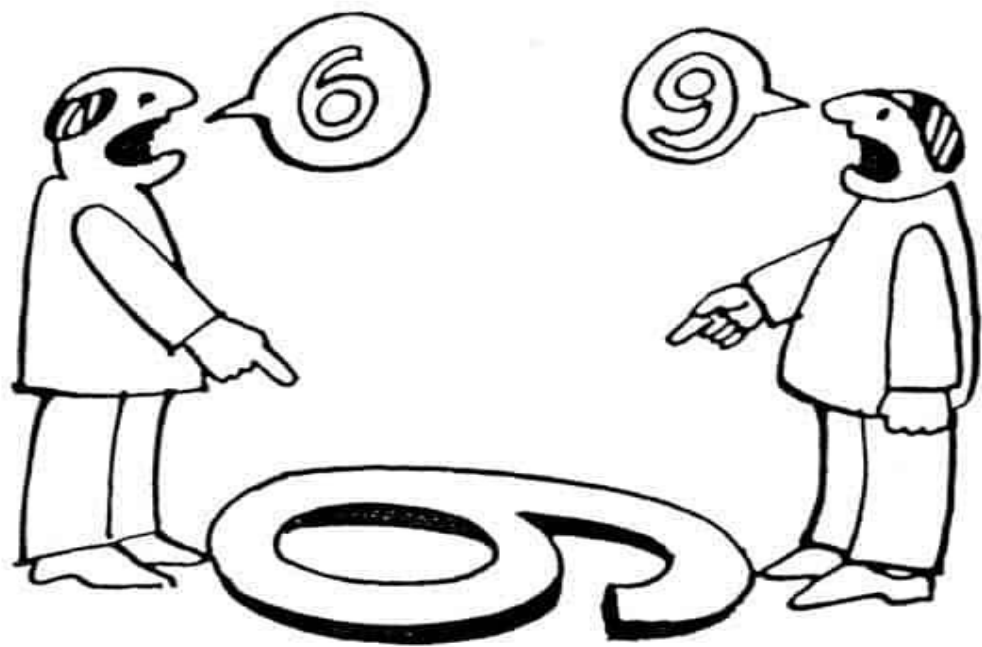


An elephant is
like a big snake

What are you
saying! It is like a
sheath of leather!!

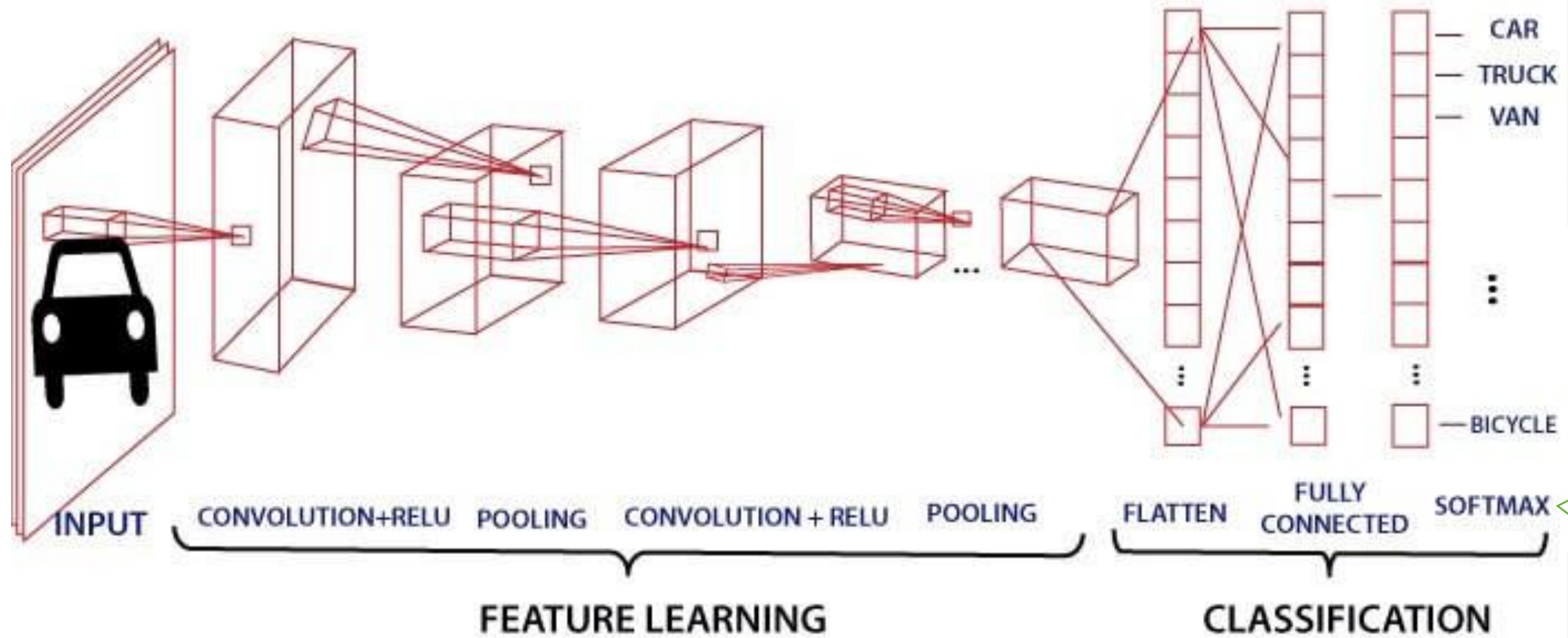
Your all wrong!!!
It's actually like a
little furry mouse.

Actually, No!
It's a tree stump!



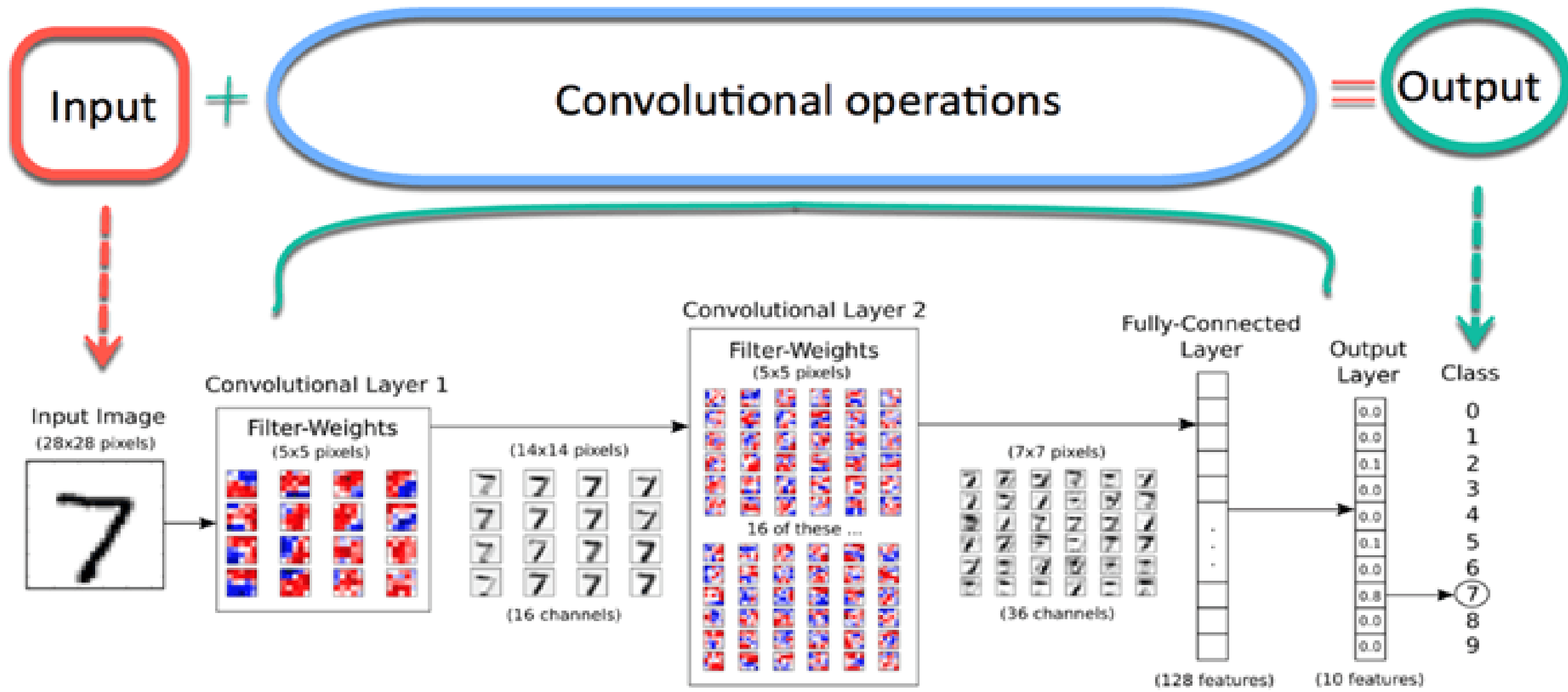


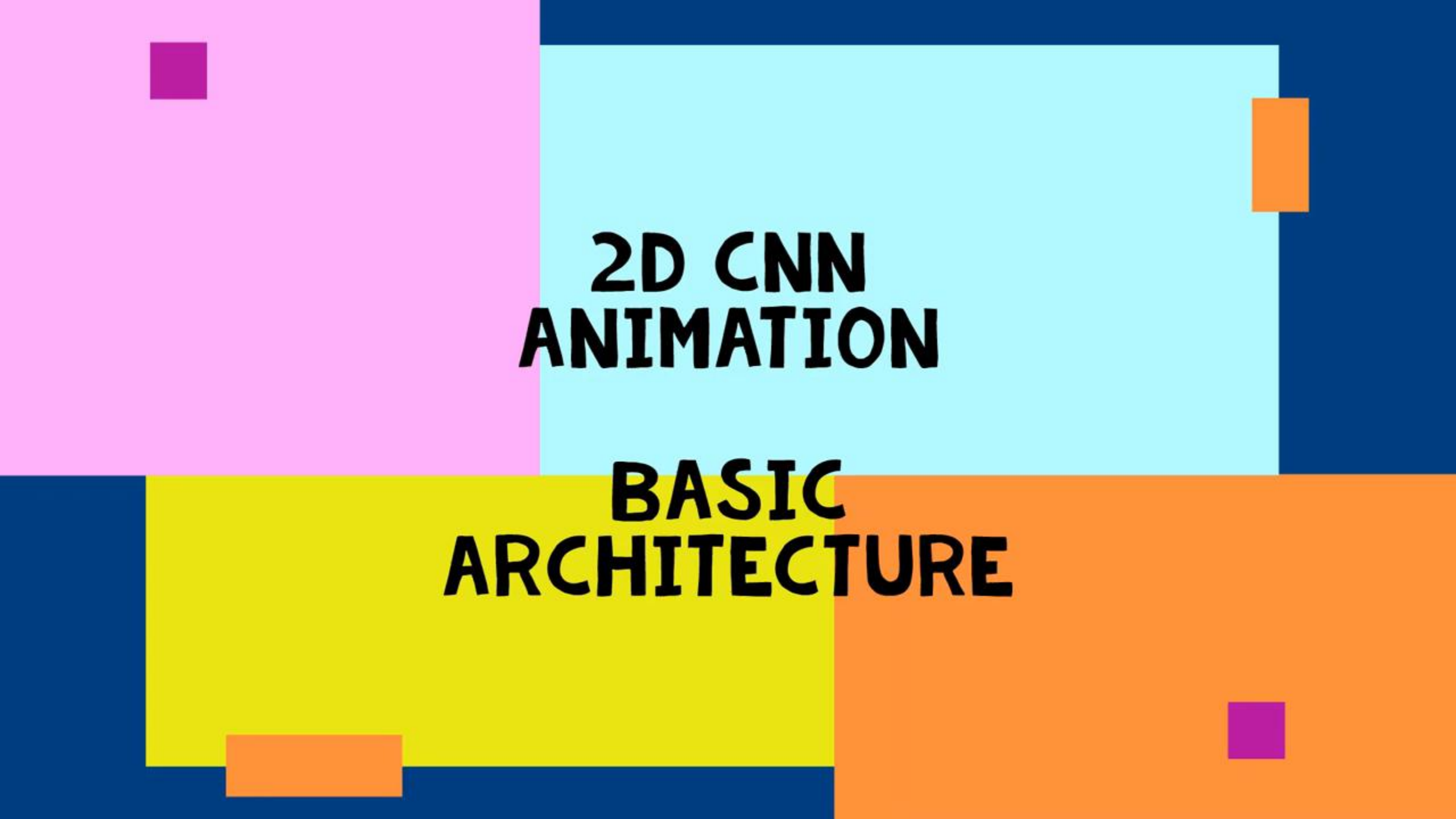
CNN





CNN





**2D CNN
ANIMATION**

**BASIC
ARCHITECTURE**

Kernel Convolution Example

Input Image

10	10	10	10	10	10
10	10	10	10	10	10
10	10	10	10	10	10
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

*

Kernel

1	2	1
0	0	0
-1	-2	-1

=

Feature Map

Original Image (Size 5x5)

7 _{x1}	2 _{x0}	3 _{x-1}	3	8
4 _{x1}	5 _{x0}	3 _{x-1}	8	4
3 _{x1}	3 _{x0}	2 _{x-1}	8	4
2	8	7	2	7
5	4	4	5	4

Filter (Size 3x3)

1	0	-1
1	0	-1
1	0	-1

*

=

Feature Map

6		

$$\begin{aligned}
 &7 \times 1 + 4 \times 1 + 3 \times 1 + \\
 &2 \times 0 + 5 \times 0 + 3 \times 0 + \\
 &3 \times -1 + 3 \times -1 + 2 \times -1 \\
 &= 6
 \end{aligned}$$

Step 1

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

=

6	-9	

$$\begin{aligned}
 &2 \times 1 + 5 \times 1 + 3 \times 1 + \\
 &3 \times 0 + 3 \times 0 + 2 \times 0 + \\
 &3 \times -1 + 8 \times -1 + 8 \times -1 \\
 &= -9
 \end{aligned}$$

Step 2



Convolutional Layer

Example 5x5 image with binary pixels

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Example 3x3 filter

1	0	1
0	1	0
1	0	1

bias

1

Feature Map

5	

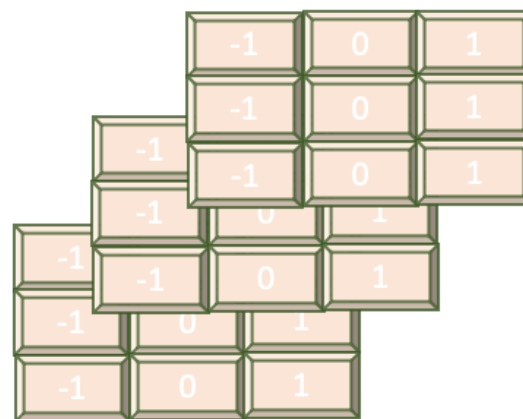
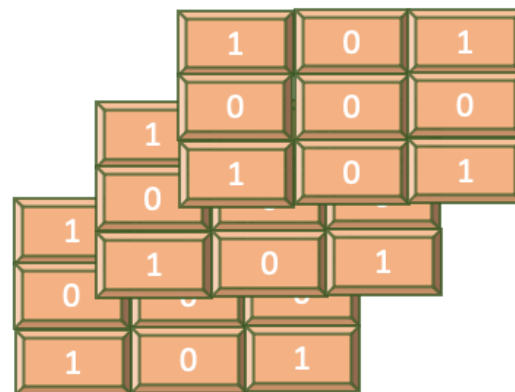
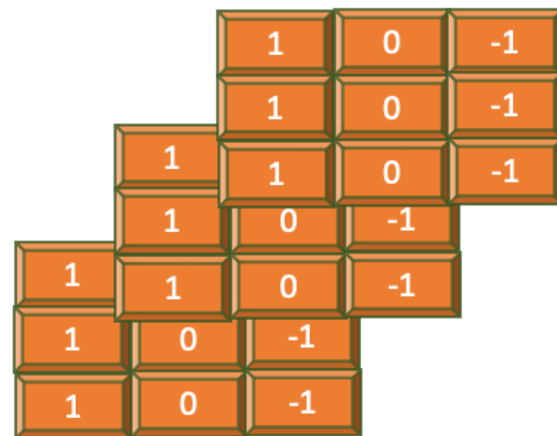
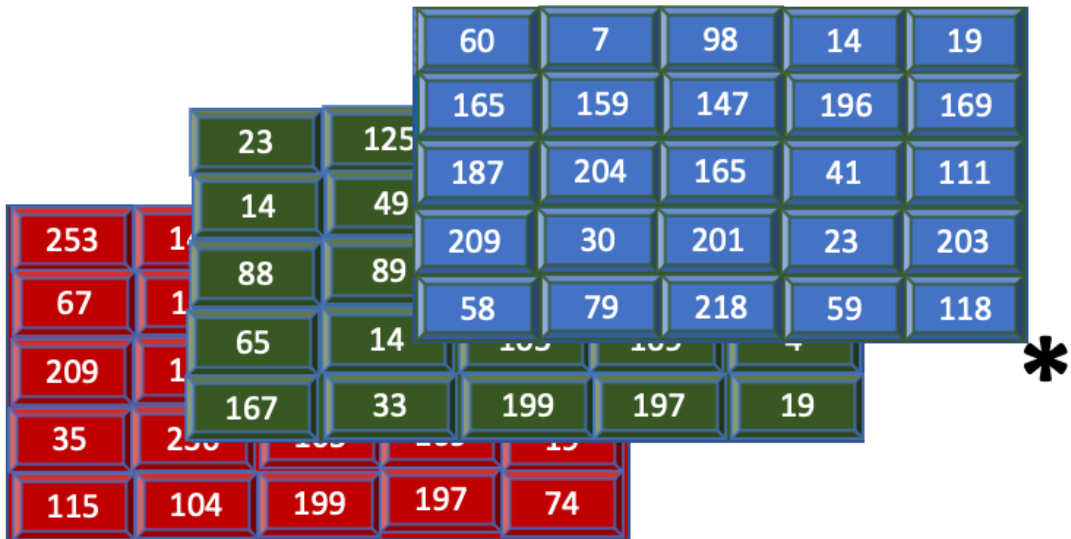
$$1 \times 1 + 1 \times 0 + 1 \times 1 + 0 \times 0 + 1 \times 1 + 1 \times 0 + 0 \times 1 + 0 \times 0 + 1 \times 1 = 4 + 1 = 5$$



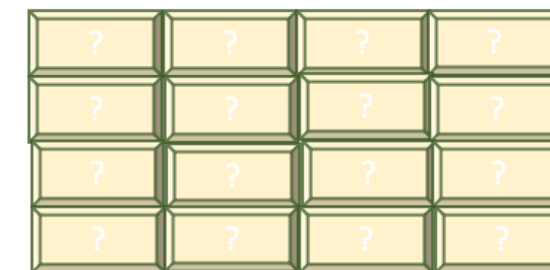
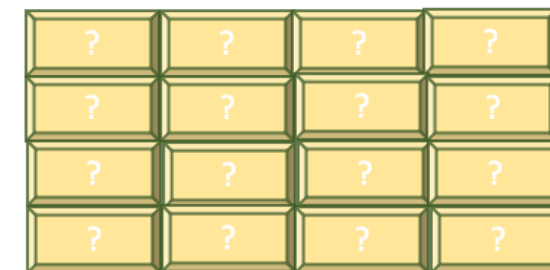
Input

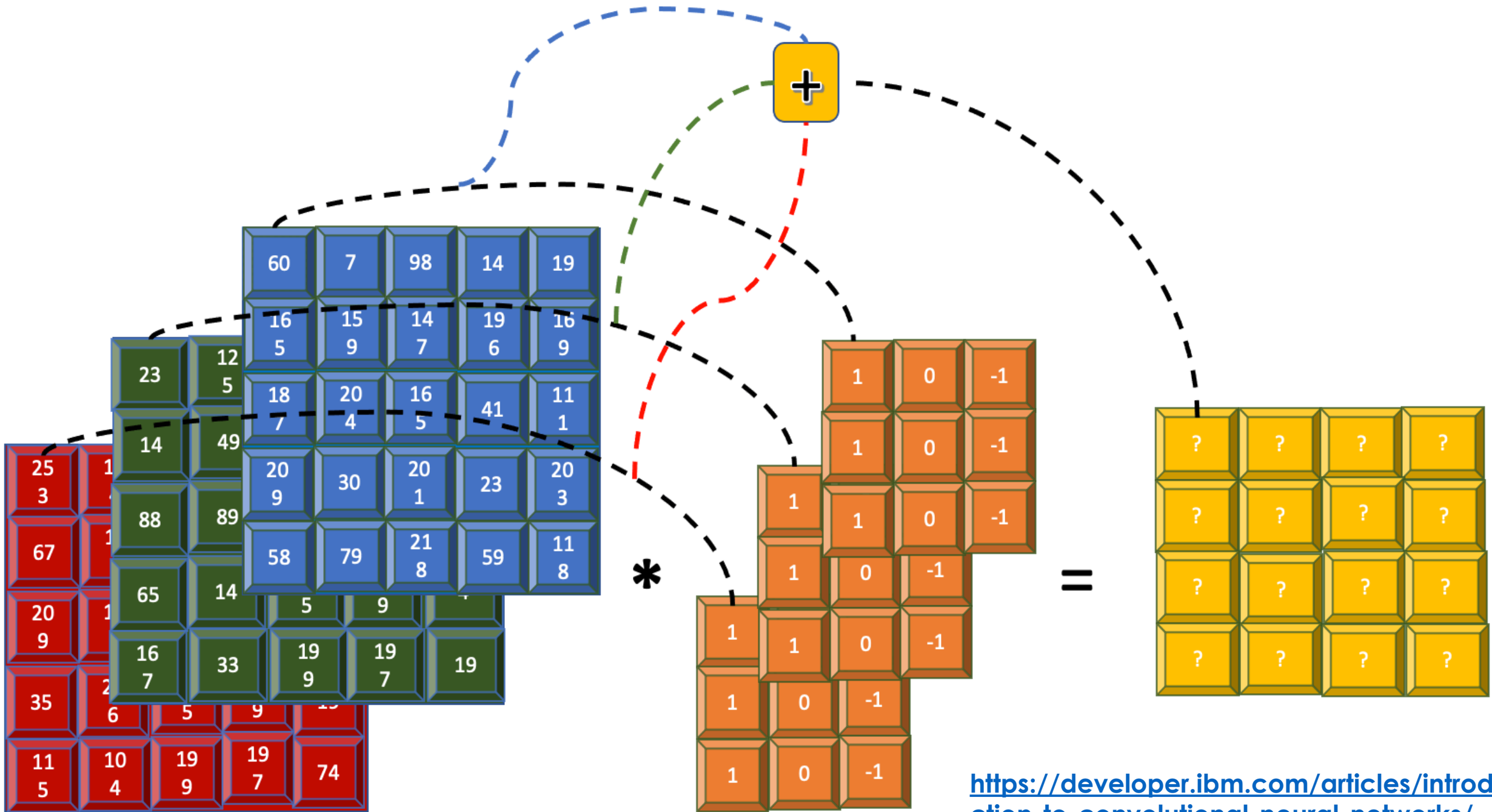


Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

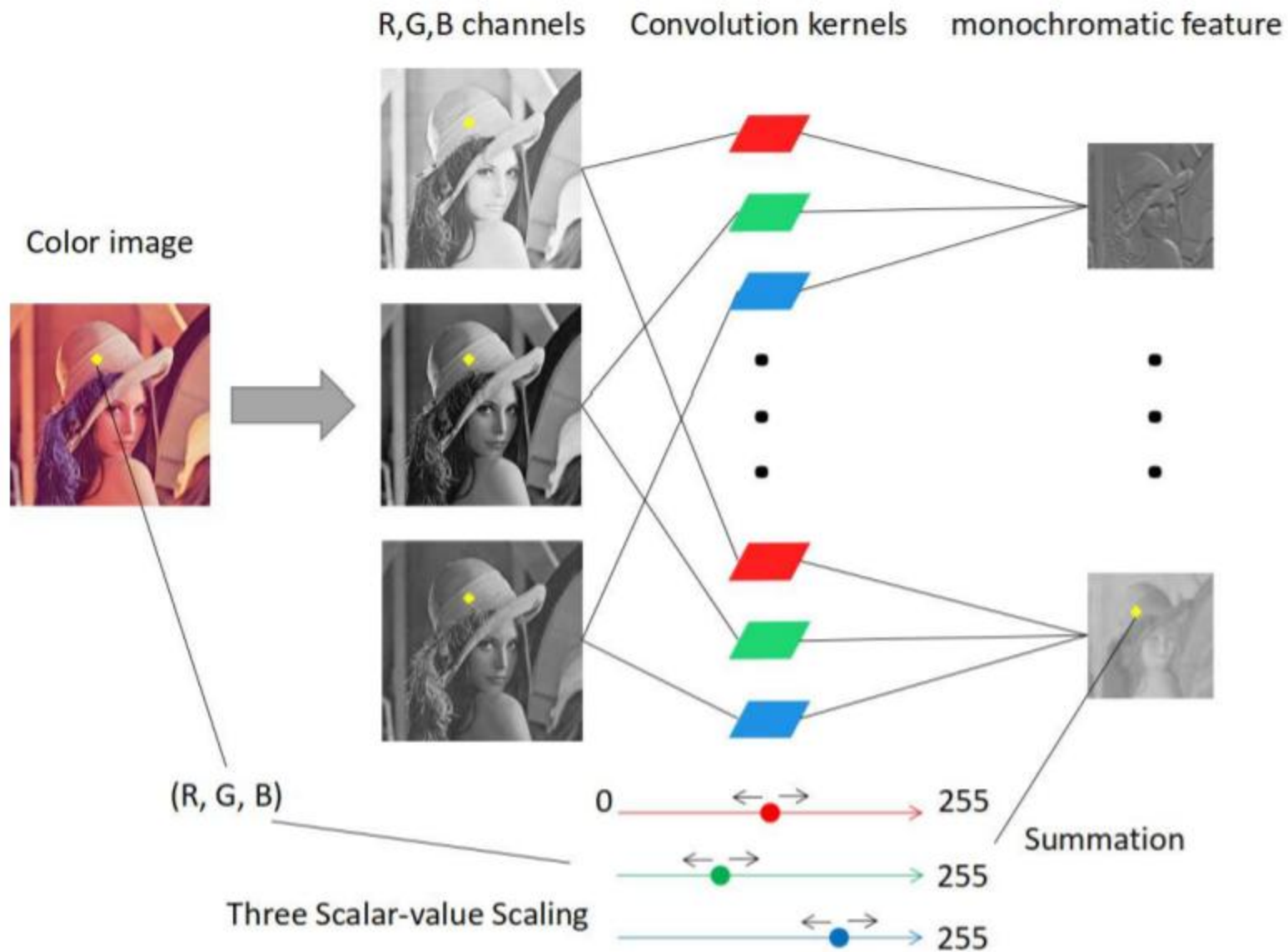


=





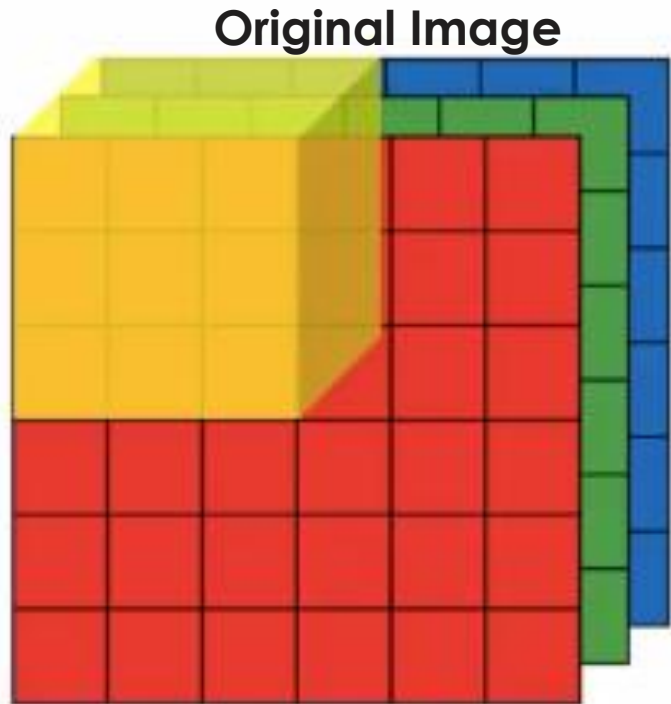
<https://developer.ibm.com/articles/introduction-to-convolutional-neural-networks/>



(a) Real-valued CNN



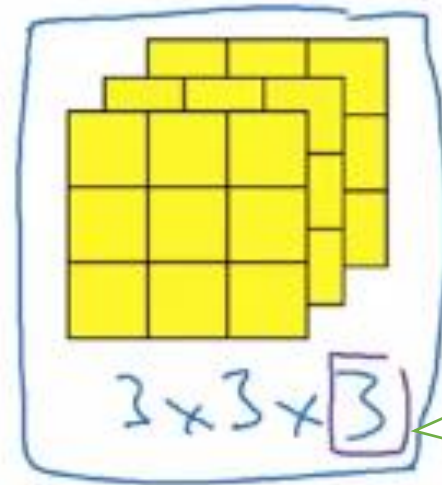
Convolutions on RGB image



$6 \times 6 \times 3$

From RGB

*



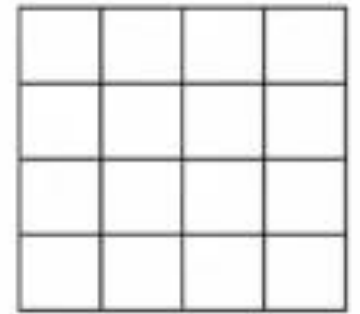
$3 \times 3 \times 3$

From RGB

=



Final Feature Map



4×4

As if there
is **x1**

Andrew Ng



PADDING AND STRIDE



Padding





Padding

0	0	0	0	0	0
0	2	1	3	5	0
0	1	1	4	4	0
0	6	2	7	3	0
0	1	6	5	5	0
0	0	0	0	0	0

Another constant number can also be used instead of zero (0).

2	2	1	3	5	5
2	2	1	3	5	5
1	1	1	4	4	4
6	6	2	7	3	3
1	1	6	5	5	5
1	1	6	5	5	5

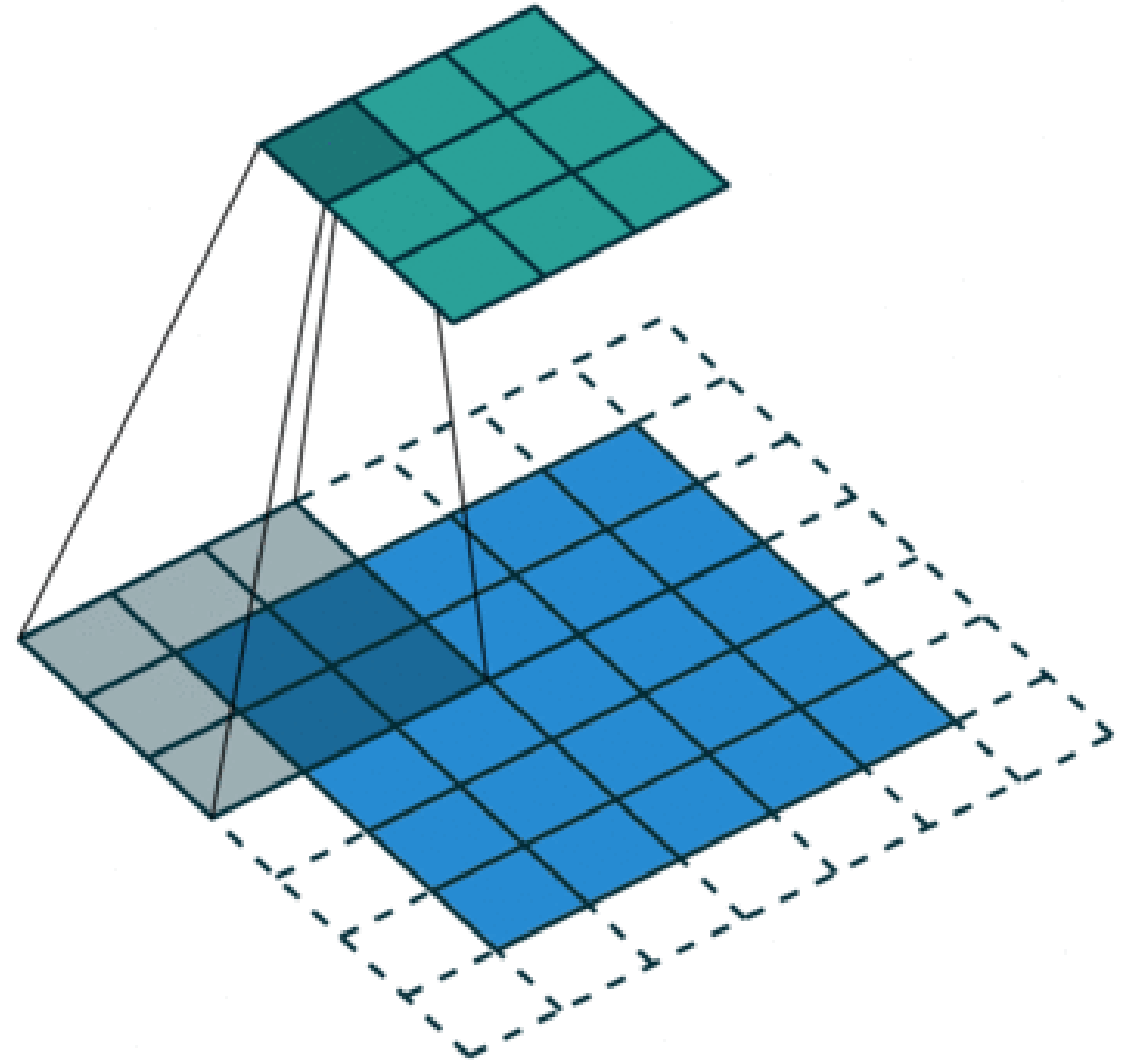


PADDING

Padding works by extending the area of which a convolutional neural network processes an image.

0 ₂	0 ₀	0 ₁	0	0	0	0
0 ₁	2 ₀	2 ₀	3	3	3	0
0 ₀	0 ₁	1 ₁	3	0	3	0
0	2	3	0	1	3	0
0	3	3	2	1	2	0
0	3	3	0	2	3	0
0	0	0	0	0	0	0

1	6	5
7	10	9
7	10	8





Convolutional Layer

Padding

Input

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

Kernel

0	1
2	3

*

=

Output

0	3	8	4
9	19	25	10
21	37	43	16
6	7	8	0



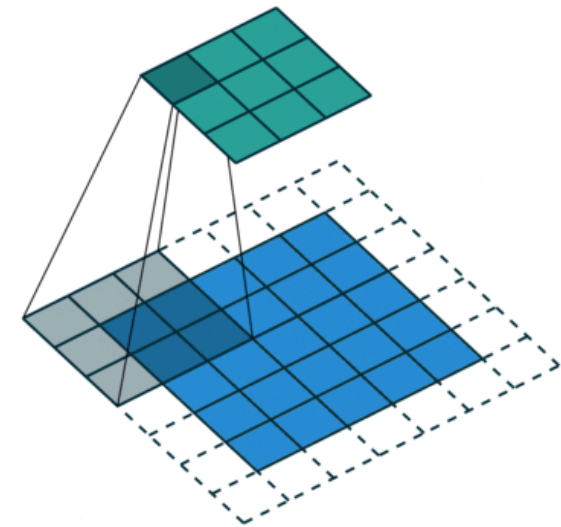
PADDING

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				



Padding: "valid"

3	5	2	7
4	1	3	8
6	3	8	2
9	6	1	5

1	2	1
2	1	2
1	1	2



55	52
57	50

Padding: "same"

0	0	0	0	0	0
0	3	5	2	7	0
0	4	1	3	8	0
0	6	3	8	2	0
0	9	6	1	5	0
0	0	0	0	0	0

1	2	1
2	1	2
1	1	2



19	26	46	22
29	55	52	40
42	57	50	43
36	46	44	19

Padding preserves the properties (size) of the data more.

Thanks to padding, the computer can better learn the information around the edges of the image.



STRIDE

Padding = Same

Filter

1	0
0	0.5

Stride X →

Input

0	0	0	0	0	0
0	1	0	0.5	0.5	0
0	0	0.5	1	0	0
0	0	1	0.5	1	0
0	1	0.5	0.5	1	0
0	0	0	0	0	0

Stride Y ↓

Output

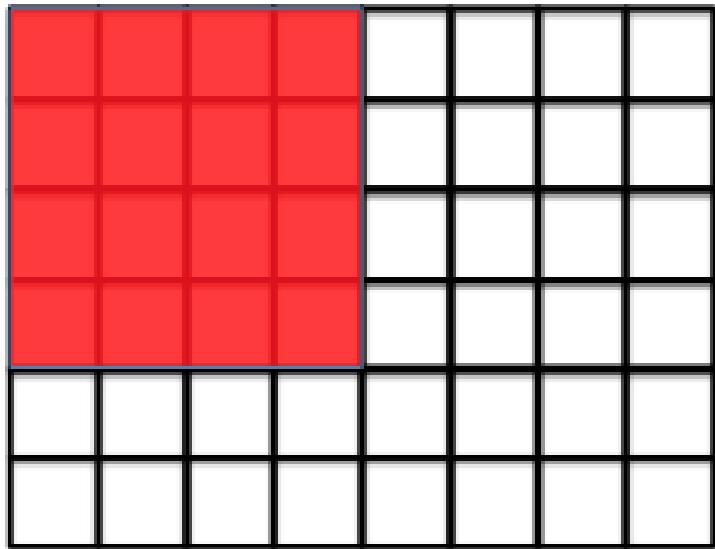
0.5	0	0.25	0.25
0	1.25	0.5	0.5
0	0.5	0.75	1.5
0.5	0.25	1.25	1

$$\text{outDim} = (\text{inpDim}) / \text{strideDim}$$



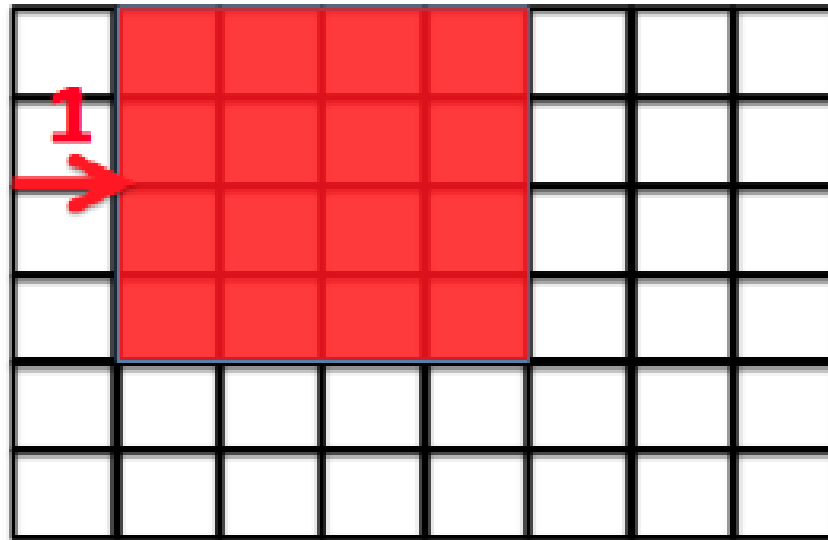
STRIDE

Filter



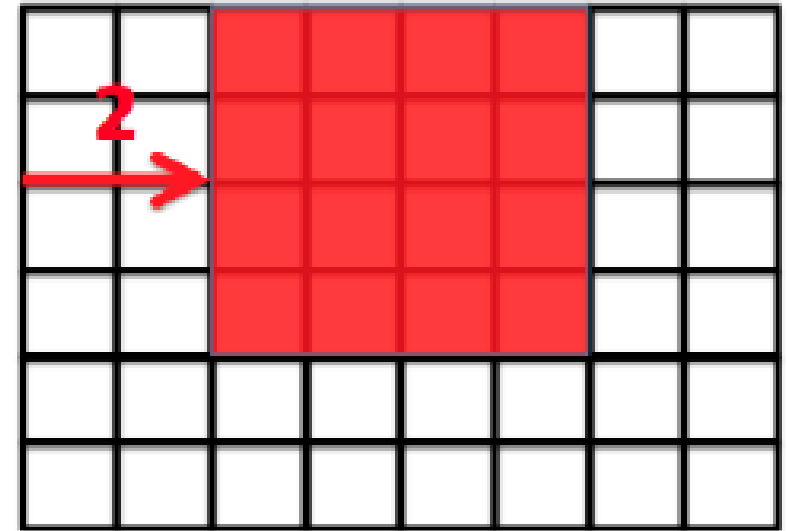
Image

Filter



Image

Filter



Image



Convolutional Layer

What will the output size be when you process a 28x28-pixel image with a 3x3 filter, 0 padding, and 1 stride?

Output Dimension= ((Input Dimension – Filter Dimension) / Stride) + 1

Output Dimension = ((28 - 3) / 1) + 1

Output Dimension = 26

Output Dimension 26x26 pixel.



NON-LINEARITY

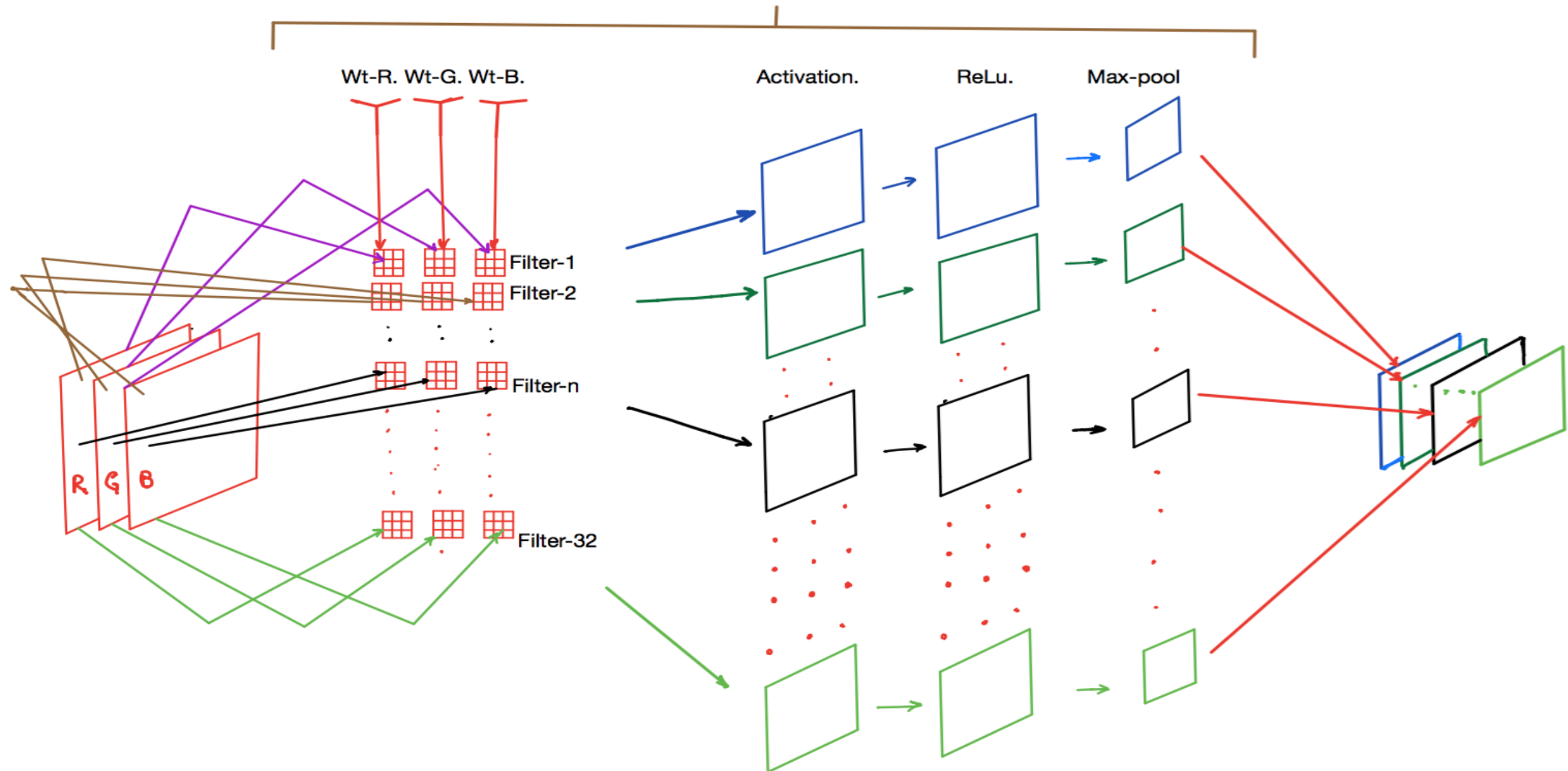


PERSONAL RECOVERY IS **NOT**
A **LINEAR** PROCESS



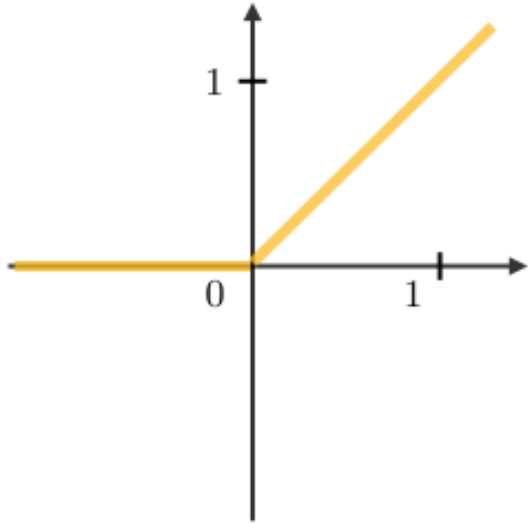
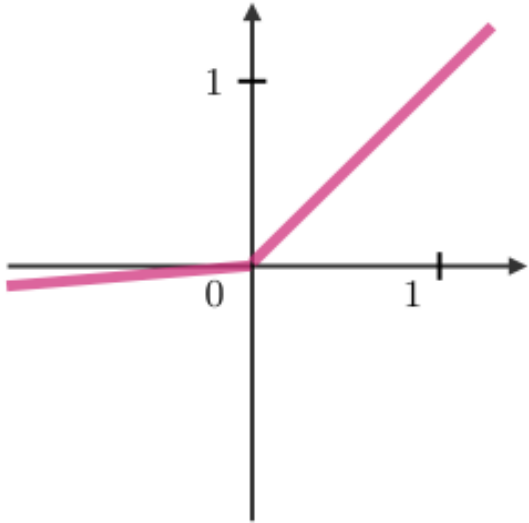
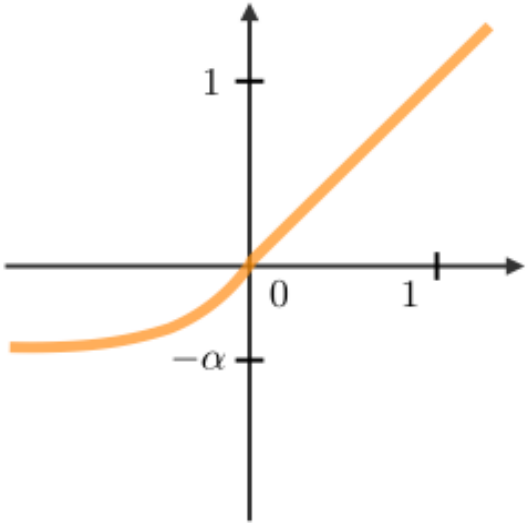
Non-linearity

First Convolution unit.



Commonly used activation functions

❑ **Rectified Linear Unit** — The rectified linear unit layer (ReLU) is an activation function g that is used on all elements of the volume. It aims at introducing non-linearities to the network. Its variants are summarized in the table below:

ReLU	Leaky ReLU	ELU
$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$	$g(z) = \max(\alpha(e^z - 1), z)$ with $\alpha \ll 1$
		
• Non-linearity complexities biologically interpretable	• Addresses dying ReLU issue for negative values	• Differentiable everywhere

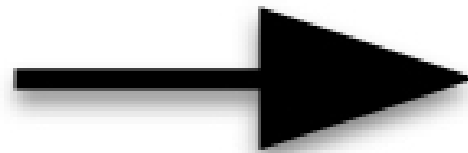


Non-linearity

ReLU Layer

Filter 1 Feature Map

9	3	5	-8
-6	2	-3	1
1	3	4	1
3	-4	5	1



9	3	5	0
0	2	0	1
1	3	4	1
3	0	5	1



Non-linearity



Original Image



Feature Map



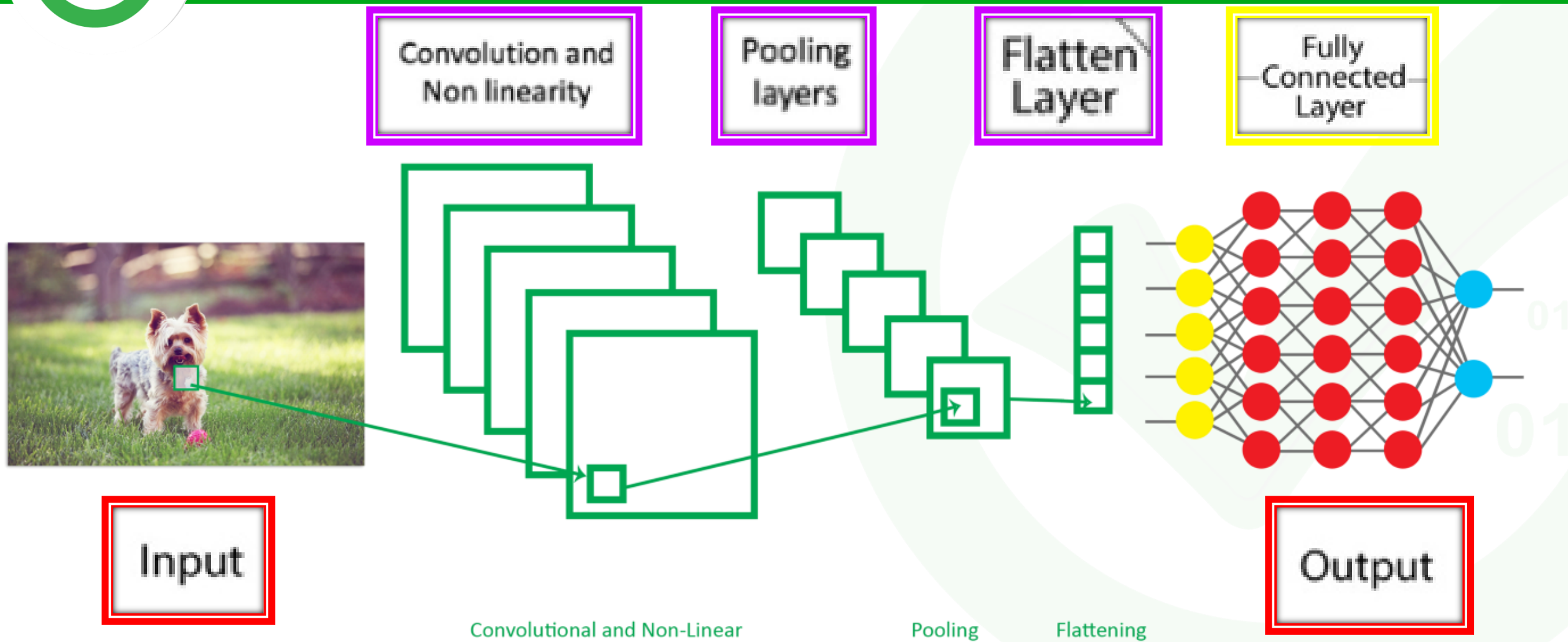
Non-Linear



POOLING LAYER

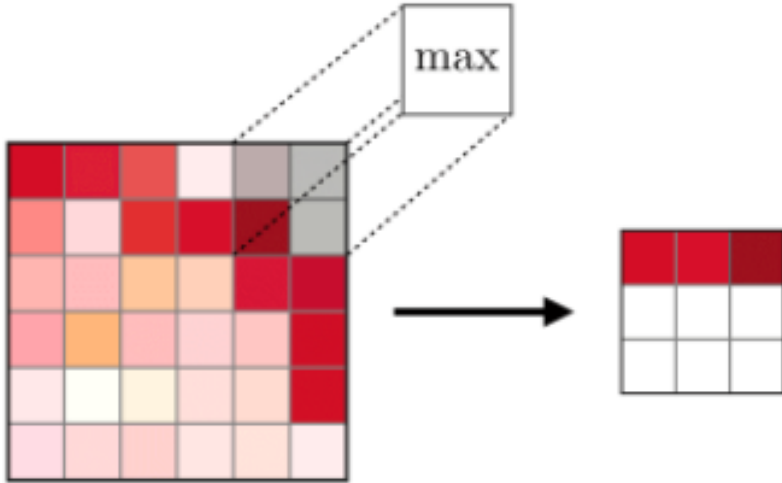
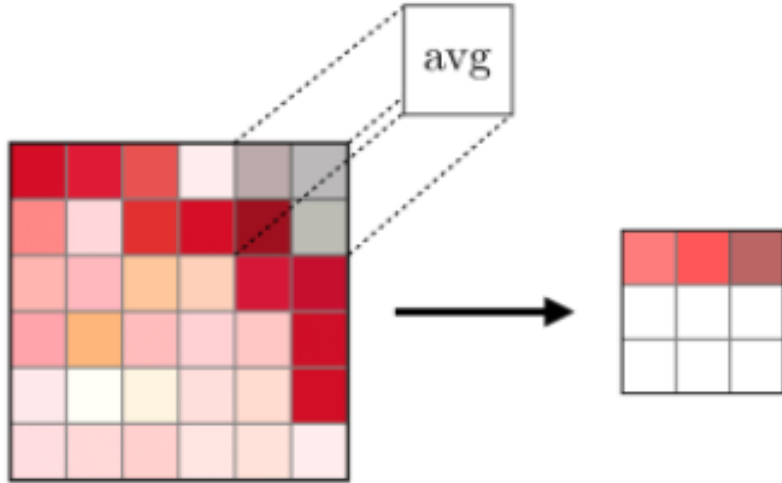


CNN





Pooling Layer

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used	• Downsamples feature map • Used in LeNet



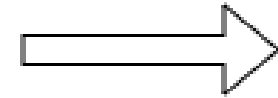
Pooling Layer

Max Pooling

4	3	1	5
1	3	4	8
4	5	4	3
6	5	9	4

$$\text{Max}([4, 3, 1, 3]) = 4$$

4	3	1	5
1	3	4	8
4	5	4	3
6	5	9	4



4	8
6	9



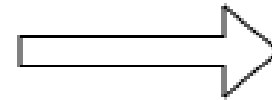
Pooling Layer

Average Pooling

4	3	1	5
1	3	4	8
4	5	4	3
6	5	9	4

$$\text{Avg}([4, 3, 1, 3]) = 2.75$$

4	3	1	5
1	3	4	8
4	5	4	3
6	5	9	4



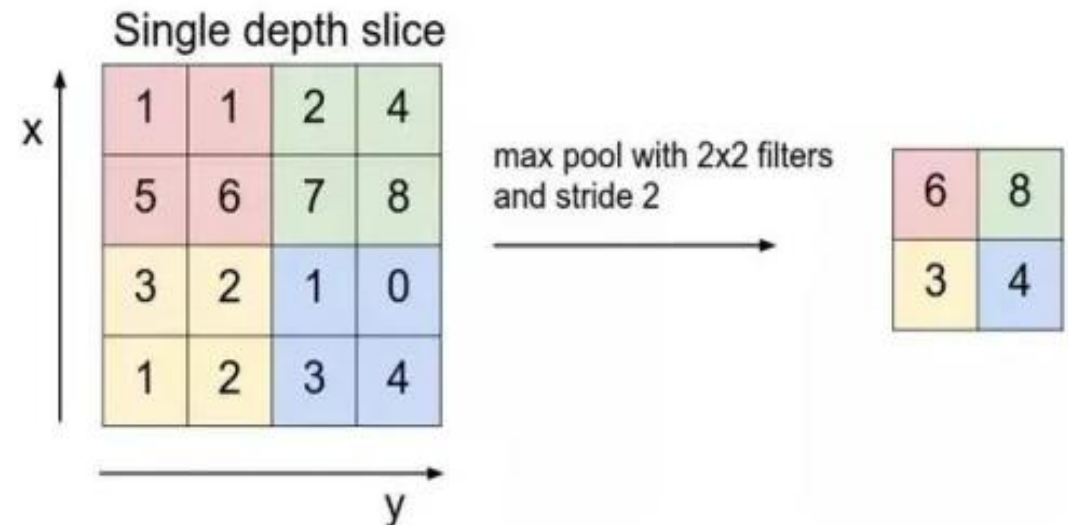
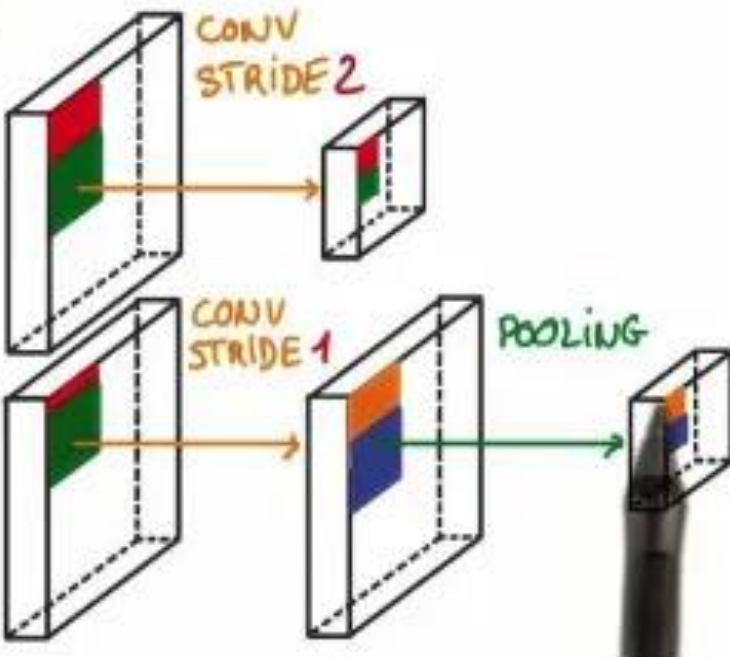
2.8	4.5
5.0	5.0



Pooling vs. Stride for Downsampling

- Pooling and stride both can be used to downsample the image.
- We usually set Stride = 1 and add a pooling layer to reduce the feature map.
- However, setting Stride = 2 and not using pooling is also a way. Computer can/may learn better (don't skip any detail) in this way.
- If you have a large image (like 1920x1080) you can use Stride = 2 and a pooling layer together.

POOLING

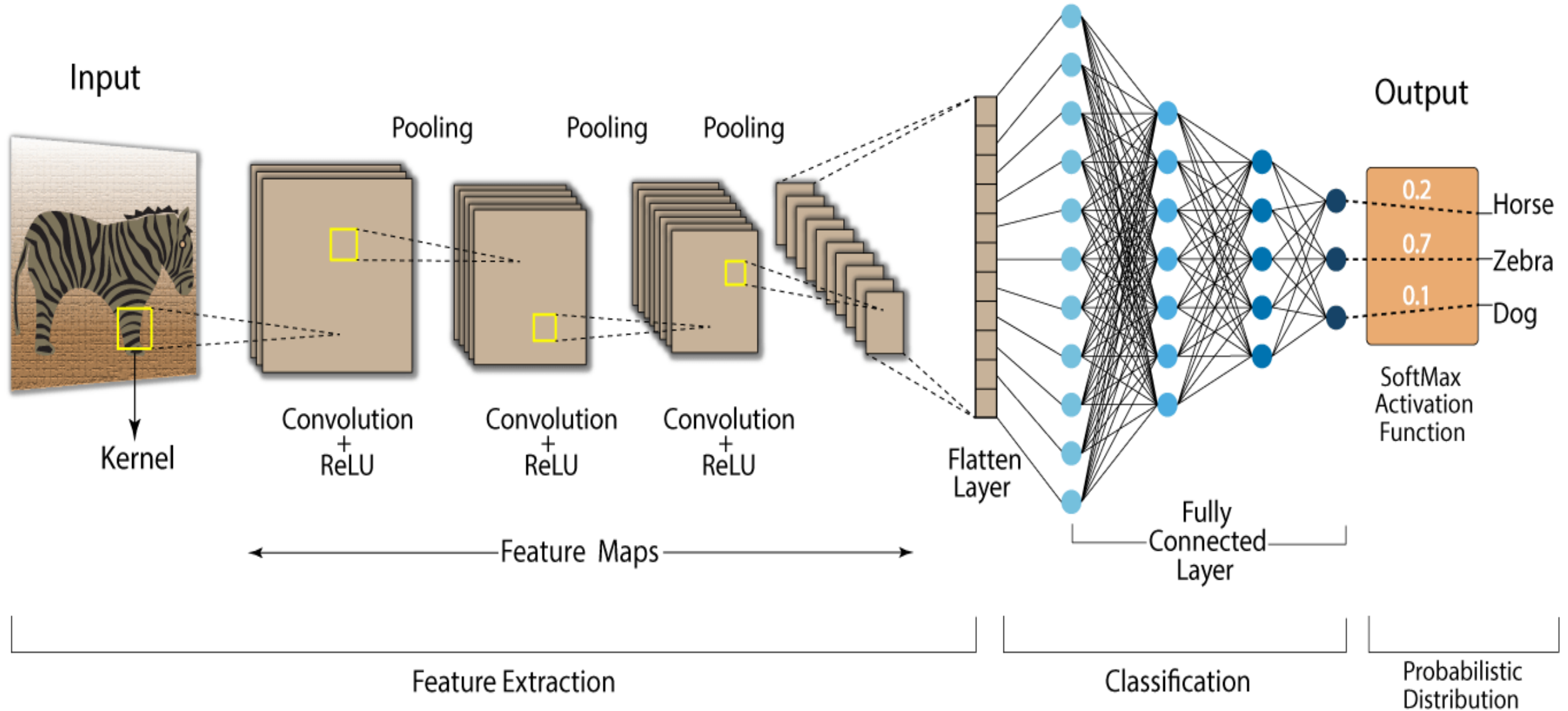


[For further information](#)



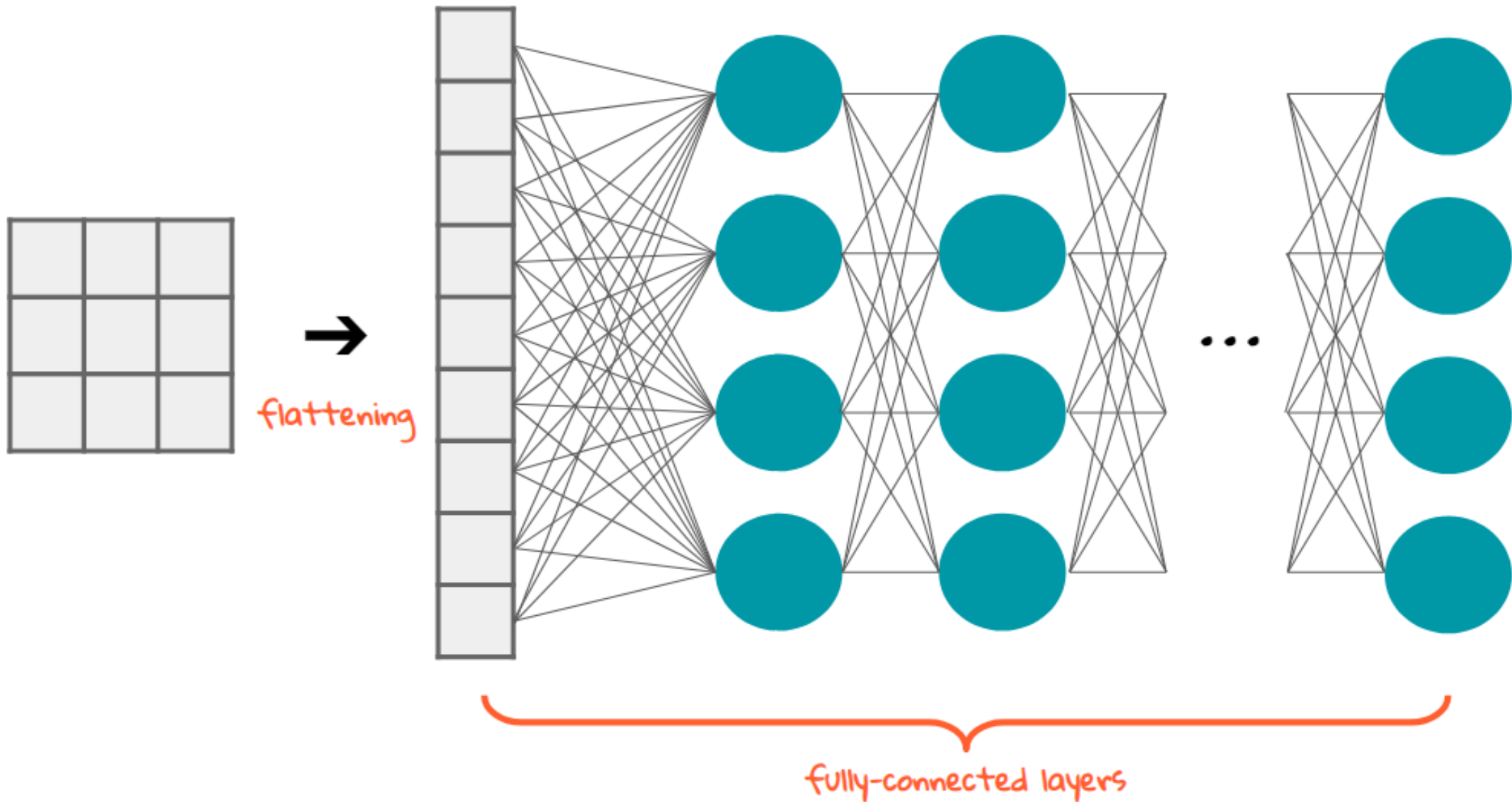
FLATTENING

Convolution Neural Network (CNN)





Flattening





Flattening

1	1	0
4	2	1
0	2	1

Pooled Feature Map

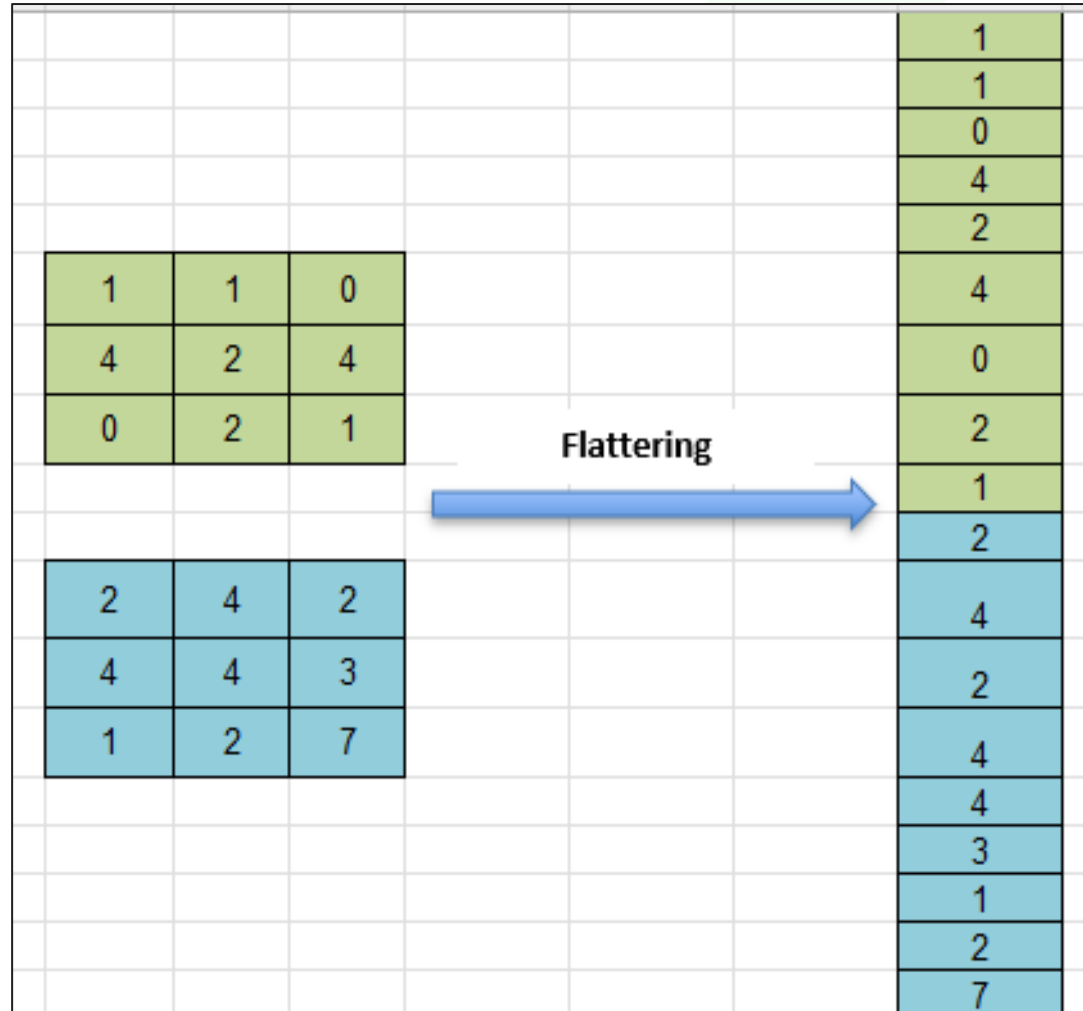
Flattening



1
1
0
4
2
1
0
2
1



Flattening

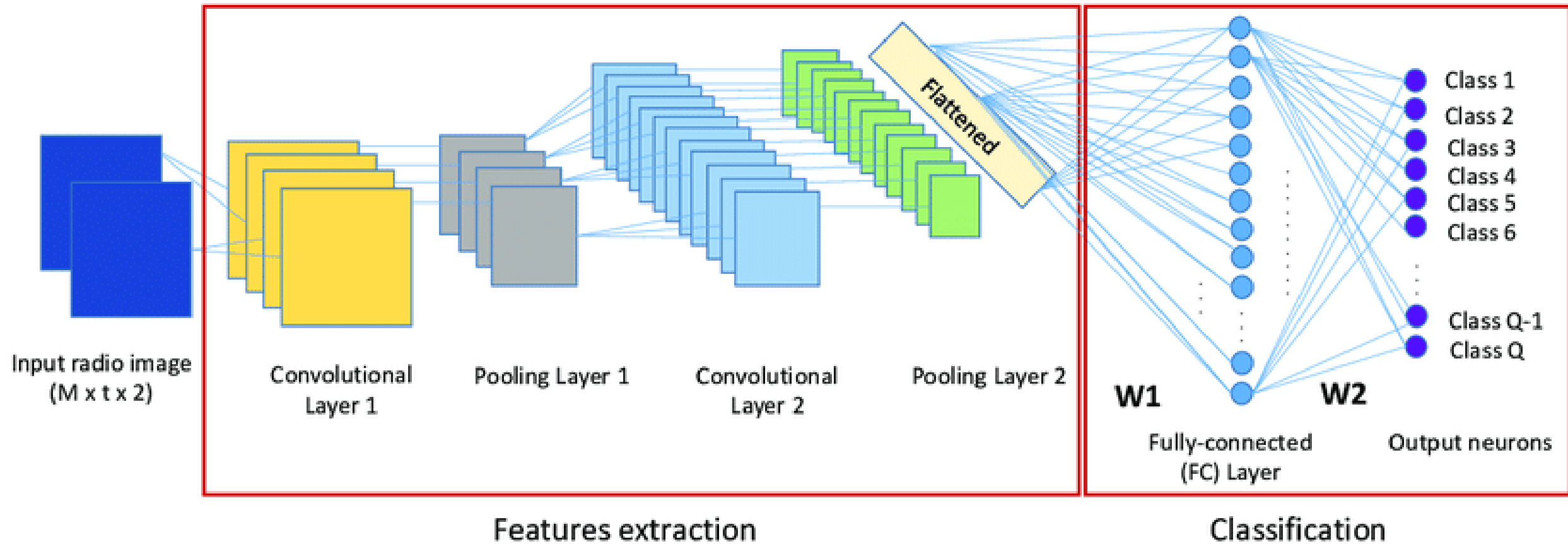




FULLY CONNECTED LAYER

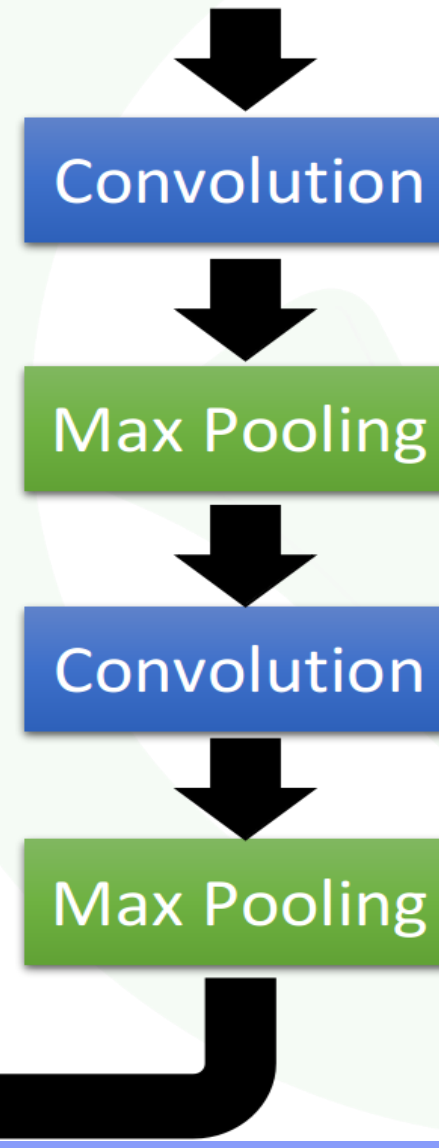
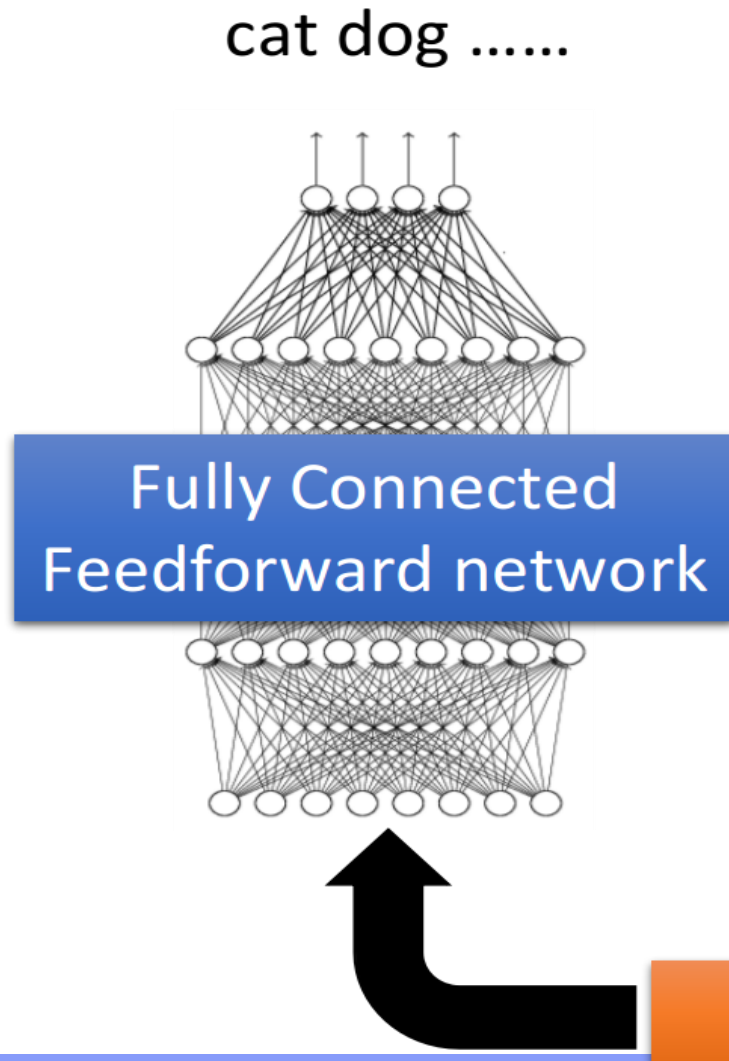


Fully-Connected Layer





The whole CNN



Can repeat many times



NOTEBOOK SAMPLES WITH CNN



CNN

Data Preprocessing

Data Preprocessing for Train Data

```
In [ ]: train_datagen = ImageDataGenerator(rescale = 1./255,  
                                          shear_range = 0.2,  
                                          zoom_range = 0.2,  
                                          horizontal_flip = True)  
training_set = train_datagen.flow_from_directory('dataset/training_set',  
                                                target_size = (64, 64),  
                                                batch_size = 32,  
                                                class_mode = 'binary')
```

Found 8000 images belonging to 2 classes.

- Here, the images are reduced to 64x64 size by applying a size reduction process.
- The parameters used here refer to parameters related to image processing.
- A large batch size allows for a more accurate gradient value to be calculated.



CNN

Data Preprocessing for Test Data

```
In [ ]: test_datagen = ImageDataGenerator(rescale = 1./255)
        test_set = test_datagen.flow_from_directory('dataset/test_set',
                                                    target_size = (64, 64),
                                                    batch_size = 32,
                                                    class_mode = 'binary')
```

Found 2000 images belonging to 2 classes.



CNN

Creation of CNN architecture

Creation an object

```
] In [ ]: cnn = tf.keras.models.Sequential()
```

Step 1: Convolution

```
] In [ ]: cnn.add(tf.keras.layers.Conv2D(filters=32, kernel_size=3, activation='relu', input_shape=[64, 64, 3]))
```

Thanks to the Conv2D function used here, the data will be handled in 64x64 pixel size. The 3 parameter here represents the RGB layers since the images are coloured.



CNN

Step 2: - Pooling

```
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2, strides=2))
```

MaxPool was used for the pooling process. Pool size = 2. Step length (stride) = 2.

These are completely up to you.

You can change them and use the model that gives you the best result.

Step 3: Second Convolutional Layer

```
cnn.add(tf.keras.layers.Conv2D(filters=32, kernel_size=3, activation='relu'))  
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2, strides=2))
```



CNN

Step 4: Flattening

```
! cnn.add(tf.keras.layers.Flatten())
```

Step 5: Full Connection

```
! cnn.add(tf.keras.layers.Dense(units=128, activation='relu'))
```

In the fully connected layer, hidden layers and neurons to be used in ANN are created. Here, there are 128 neurons and ReLu was used as the activation function. Since the images used are 64x64, the number of neurons is taken as 128 and you can change it.

Step 6: Output Layer

```
! cnn.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))
```

- The "units" in the output layer shows the output number. If the output is 1 here, it means dog, if it is 0, it means cat. We can use "sigmoid" at binary classification problems. In multiclass classification problems, the class number is written here.
- "Activation" function changes model outputs with probability. We can use "sigmoid" at **binary classification** problems and "softmax" at **multiclass classification** problems.
- In **regression** problems, units = 1 and the activation function is not added. `model.add(tf.keras.layers.Dense(1))`



CNN

Compile

```
cnn.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['accuracy'])
```

In the last step, the "adam" function was used for the optimizer from the output layer. "accuracy" was selected as the metrics.

Train the Model

```
cnn.fit(x = training_set, validation_data = test_set, epochs = 50)
```

It will train the model using the "training_set" data. It will work with 50 repetitions while doing this process. Then it will test model on the "test_set" data.

```
Epoch 3/50  
250/250 [=====] - 44s 176ms/step - loss: 0.5871 - accuracy: 0.6883 - val_loss: 0.5813 - val_accu  
racy: 0.6940  
Epoch 4/50  
250/250 [=====] - 43s 173ms/step - loss: 0.5530 - accuracy: 0.7074 - val_loss: 0.5068 - val_accu  
racy: 0.7520  
Epoch 5/50  
250/250 [=====] - 46s 183ms/step - loss: 0.5105 - accuracy: 0.7457 - val_loss: 0.4880 - val_accu  
racy: 0.7700  
Epoch 6/50  
250/250 [=====] - 47s 189ms/step - loss: 0.4890 - accuracy: 0.7599 - val_loss: 0.5459 - val_accu  
racy: 0.7355  
Epoch 7/50  
250/250 [=====] - 43s 173ms/step - loss: 0.4754 - accuracy: 0.7702 - val_loss: 0.4821 - val_accu
```



CNN

Get prediction by using a sample image from your local (computer)

```
import numpy as np
from keras.preprocessing import image
test_image = image.load_img('dataset/single_prediction/cat_or_dog_1.jpg', target_size = (64, 64))
test_image = image.img_to_array(test_image)
test_image = np.expand_dims(test_image, axis = 0)
result = cnn.predict(test_image)
training_set.class_indices
if result[0][0] == 1:
    prediction = 'dog'
else:
    prediction = 'cat'
```

```
print(prediction)
```

dog



A great [website](#) explaining CNN's working logic.

